

Z3 Internals (Draft)

Nikolaj Bjørner Clemens Eisenhofer Arie Gurfinkel
Microsoft Research TU Wien U Waterloo

Nuno P. Lopes Leonardo de Moura Lev Nachmanson
U Lisbon Microsoft Research Microsoft Research

Christoph Wintersteiger
Microsoft Research

Abstract. We present an overview of Z3 *internals* to outline the main data-structures and algorithms used in Z3. The overarching aim is to help users gain more insight into *formula encodings that have a better chance of being solved*, *what controls a user has over search behavior*, and *what information a user can extract from the solver*. While we will try to address these questions by providing background on principles and examples of experiences, it is impossible to provide comprehensive answers. The hope is that shedding light into the internal workings of Z3 at a conceptual level offers a bridge towards developing an understanding of how to operate Z3 as well.



Contents

1. **Introduction**
 - 1.1. Resources
2. **Solver Cores**
 - 2.1. CDCL(T) - SAT in the light of Theory Solvers
 - 2.1.1. Other Inference Rules
 - 2.1.2. Invariants
 - 2.2. NLSAT (MC-SAT)
 - 2.3. SPACER
 - 2.4. QSAT
3. **Decision Procedures**
 - 3.1. Boolean Theories
 - 3.2. Equality and Uninterpreted Functions
 - 3.2.1. E-Node
 - 3.2.2. Merge
 - 3.2.3. Unmerge
 - 3.2.4. Justifications
 - 3.2.5. From justifications to proofs
 - 3.2.6. Integration with Boolean reasoning
 - 3.3. Arithmetic
 - 3.3.1. Rational linear arithmetic
 - 3.3.2. Integer linear arithmetic
 - 3.3.3. Non-linear arithmetic
 - 3.3.4. Data-structures
 - 3.4. Reducible Theories
 - 3.4.1. Refinement Types
 - 3.4.2. Arrays
 - 3.4.3. Algebraic Datatypes
 - 3.5. Hybrid Theories
 - 3.5.1. Sequences/Strings/Regex
 - 3.5.2. Floating points
4. **Data-structure and algorithm internals**
 - 4.1. Terms and Formulas
 - 4.2. Representing assertions
 - 4.3. Compiling terms and formulas into clauses and E-nodes
 - 4.4. Interaction with the CDCL(T) core
 - 4.5. Theory Lemmas, Axioms and Propagations
 - 4.6. Model-based theory combination
 - 4.7. Relevancy filtering
 - 4.8. The power of propagation
 - 4.9. Dynamic Ackermann reduction
 - 4.10. Iterative deepening for finite models
 - 4.11. Incrementality
5. **Model Construction**
 - 5.1. Extracting models from satisfiable states
 - 5.2. Characteristics of Models
 - 5.2.1. Models are explicit
 - 5.2.2. Models are mainly based on values
 - 5.2.3. There are no deep knobs to control model values
 - 5.2.4. Models include auxiliary definitions
6. **Certificates and Unsatisfiable Cores**

- 6.1. Unsatisfiable Cores
- 6.2. Certificates
 - 6.2.1. Legacy Proof Format
 - 6.2.2. Proof Logs
 - 6.2.3. EUF Axioms
 - 6.2.4. Arithmetic Axioms
- 7. Quantifiers
 - 7.1. E-matching [19]
 - 7.1.1. E-matching - basic algorithm
 - 7.1.2. E-matching - code trees
 - 7.1.3. Inverted path indexing
 - 7.1.4. Relevance Propagation
 - 7.1.5. The life-cycle of E-matching.
 - 7.1.6. Prioritizing Quantifier Instantiation
 - 7.2. Model-based Quantifier Instantiation [10, 20, 26, 45]
 - 7.2.1. EPR
 - 7.2.2. UFBV [45]
 - 7.2.3. Map Property Fragment [11]
 - 7.2.4. Array Property Fragment [11]
 - 7.2.5. Essentially Uninterpreted Fragment [26]
 - 7.3. Quantifier Elimination through Macro Introduction
 - 7.3.1. Tactic macro-finder
 - 7.3.2. Tactic elim-predicates
- 8. Tactics
 - 8.1. A use case of tactics from Alive2
 - 8.2. A classification of tactics
 - 8.3. Model Converters
 - 8.4. Pre-processing tactics
 - 8.5. Proof and unsatisfiable core converters
 - 8.6. Incremental tactics - a sneak peek
- 9. Interfacing with Z3
 - 9.1. Feeding formulas into Z3
 - 9.2. Controlling search and output behavior
 - 9.2.1. Limitations of CDCL
 - 9.2.2. Ackermann reductions
 - 9.2.3. Converting arithmetic to propositional reasoning
 - 9.2.4. Arithmetic
 - 9.2.5. Programming Quantifiers
 - 9.3. Extracting information from Z3
- 10. Summary
- References



1. Introduction

This document is written from the perspective of a developer of the SMT solver Z3. While it attempts to introduce an audience that includes many users to *how to build an SMT solver*, the main interest among users is more likely about *how to best use an SMT solver*. By introducing the internals of Z3 from the point of view of foundations, data-structures, algorithm design, and heuristics we introduce the reader to the development process. But, in this process, we also indirectly aim to address readers whose aim is to understand the how and why of using Z3. Many of the principles underlying the implementation of Z3 have been described in papers over several years and this document is not a substitute for these papers. We therefore often resort to references when material is covered elsewhere and instead attempt to reflect updated systems aspects.

Figure 1 shows a bird's eye view of the main components of Z3.

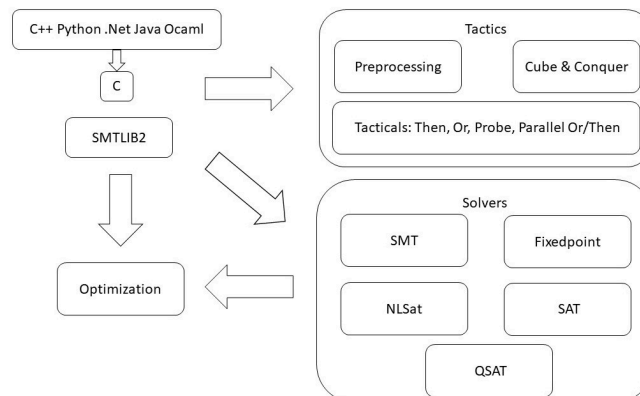


Figure 1. Overall system architecture of Z3

1.1. Resources

The main point of reference for Z3 is the GitHub repository

<https://github.com/z3prover/z3>

This document builds on top of and extends the material presented in [9] as part of a shorter tutorial.

You can use Z3 online from your browser. No installation is required

<https://microsoft.github.io/z3guide>

This guide complements the tutorial

<https://z3prover.github.io/papers/programmingz3.html>

Some of the internals are already described in depth in the *Programming Z3* tutorial so they are covered in less detail here.

2. Solver Cores

Z3 contains a collection of core engines for solving formulas. The engine used by default is based on the CDCL(T) architecture, but other engines are available and invoked for special classes of formulas where they are more suitable.

2.1. CDCL(T) - SAT in the light of Theory Solvers

We here describe the solvers marked as SAT and SMT in Figure 1.

When SAT solving, as implemented using conflict driven clause learning, CDCL, is combined with theory solving it augments propositional satisfiability with theory reasoning. The CDCL solver maintains a set F of formulas and a partial assignment to literals in F that we refer to as M .

The solver starts in a state $\langle M, F \rangle$, where M is initially empty. It then attempts to complete M to a full model of F to show that F is satisfiable and at the same time adds consequences to F to establish that F is unsatisfiable. The transition between the search for a satisfying solution and a consequence is handled by a *conflict resolution* phase. The state during conflict resolution is a triple $\langle M; F; C \rangle$, where, besides the partial model M and formula F , there is also a conflict clause C false under M . The auxiliary function *Theory* is used to advance decisions, propagations and identify conflicts. If *Theory* determines that F is conflicting with respect to the literals in M it produces a conflict clause C , that contains a subset of conflicting literals from M . It can also produce a trail assignment A , which is either a propagation or decision and finally, if it determines that F is satisfiable under trail M it produces *SAT*.

From the point of view of the CDCL(T) solver, theory reasoning is a module that can take a state during search and produce verdicts on how search should progress. We use the following verdicts of a theory invocation $Theory(M; F)$:

- *SAT*. The theory solver may determine that the assignment M extends to a model of F .
- Conflict C . The theory solver may determine that a subset M is inconsistent relative to F . In the propositional case, an inconsistent clause C is a member of F , such that each literal in C is false in M . With theory reasoning, C does not need to correspond to a clause in F , but be assignments in M that are inconsistent modulo theories.
- A propagation ℓ^C . The theory solver propagates a literal ℓ .
- A decision ℓ^δ .

Thus, the verdict of *Theory* determines whether the partial model extends to a model of the theories, can identify a subset of M as an unsatisfiable core, propagate the truth assignment of a literal ℓ , or create a new case split ℓ^δ for a literal ℓ that has not already been assigned in M . We write $SAT = Theory(M, F)$ when the verdict is that M extends to a valid theory model of F , we write $C = Theory(M, F)$ when C is a conflict clause, based on negated literals from M and $A = Theory(M, F)$, when the verdict A is either a propagation or decision.



Sat	$\langle M; F \rangle$	$\Rightarrow SAT$	$SAT = Theory(M, F)$
Conflict	$\langle M; F \rangle$	$\Rightarrow \langle M; F; C \rangle$	$C = Theory(M, F)$
Augment	$\langle M; F \rangle$	$\Rightarrow \langle M, A; F \rangle$	$A = Theory(M, F)$
Unsat	$\langle \emptyset; F; C \rangle$	$\Rightarrow UNSAT$	
Resume	$\langle M, \ell^\delta; F; C \rangle$	$\Rightarrow \langle M, \ell^C; F \rangle$	$\ell \in C$
Resolve	$\langle M, \ell^{C'}; F; C \rangle$	$\Rightarrow \langle M; F; (C \setminus \{\bar{\ell}\}) \cup (C' \setminus \{\ell\}) \rangle$	$\bar{\ell} \in C$
Backtrack	$\langle M, A; F; C \rangle$	$\Rightarrow \langle M; F; C \rangle$	<i>otherwise</i>

Example 1.

Consider the formula $(x > 0 \vee y > 0) \wedge x + y < 0$. The initial state of search is

$$\langle \emptyset; (x > 0 \vee y > 0) \wedge x + y < 0 \rangle$$

based on the empty partial assignment $\emptyset$ and the original formula. A possible next state is to propagate on the unit literal $x + y < 0$, producing

$$\langle x + y < 0^{x+y<0}; (x > 0 \vee y > 0) \wedge x + y < 0 \rangle$$

This step may be followed by a case split setting $x > 0$ to false.

$$\langle x + y < 0^{x+y<0}, \neg(x > 0)^\delta; (x > 0 \vee y > 0) \wedge x + y < 0 \rangle$$

which triggers a unit-propagation

$$\langle x + y < 0^{x+y<0}, \neg(x > 0)^\delta, y > 0^{x>0 \vee y>0}; (x > 0 \vee y > 0) \wedge x + y < 0 \rangle$$

The resulting state is satisfiable in the theory of arithmetic. On the other hand, if we had chosen to set $x > 0$ to true as well as $y > 0$, we would have encountered a conflicting state with conflict clause $\neg(x > 0) \vee \neg(y > 0)$ ¹:

$$\langle x + y < 0^{x+y<0}, x > 0^\delta, y > 0^\delta; (x > 0 \vee y > 0) \wedge x + y < 0; \neg(x > 0) \vee \neg(y > 0) \rangle$$

The last decision is then reverted to produce the satisfiable state

$$\langle x + y < 0^{x+y<0}, x > 0^\delta, \neg(y > 0)^{\neg(x>0) \vee \neg(y>0)}; (x > 0 \vee y > 0) \wedge x + y < 0 \rangle$$

A third scenario uses theory propagation. In this scenario, the decision $x > 0$ is made, but instead of making a decision $y > 0$, the theory solver for arithmetic is given a chance to find opportunities for propagation. It deduces that $x + y < 0$ together with $x > 0$ implies $\neg(y > 0)$, and therefore establishes the theory propagation

$$\langle x + y < 0^{x+y<0}, x > 0^\delta, \neg(y > 0)^{\neg(y>0) \vee \neg(x>0)}; (x > 0 \vee y > 0) \wedge x + y < 0 \rangle$$

We are again eliding the unit literal $x + y < 0$ from the explanation for $\neg(y > 0)$. In practice, solvers automatically remove literals from conflict clauses that are necessarily false.

2.1.1. Other Inference Rules

In the initial overview of the CDCL(T) loop, we left out other inference rules that are integral to efficient search.

- Learn: Conflict clauses are added to the set of formulas F . Conflict clauses are marked as redundant.
- Restart: A suffix of the current partial model M is abandoned.
- In-processes: Clauses are inferred or marked redundant based on global inferences. This is usually done after a restart.
- Garbage Collection: Redundant clauses are removed from F .

2.1.2. Invariants

To be well-behaved we expect *Theory* to produce propagations on literals that don't already appear in M , and crucially enforce the main invariants:

- The conflict clause C is false in M and a consequence of F . Thus, for state $\langle M; F; C \rangle$ we have $F \models_T C$ as well as $\bar{C} \in M$.
- A propagated literal is justified by the current partial model M . Thus, for state $\langle M, \ell^C; F \rangle$ we have $F \models_T C$, $\ell \in C$, and for each $\ell' \in C \setminus \{\ell\}$: $\bar{\ell}' \in M$.

That is, each conflict clause is a consequence of F and each propagation is also a consequence of F , and the premises of a propagation is justified by T .

2.2. NLSAT (MC-SAT)

Z3 contains a dedicated engine for non-linear arithmetic over polynomials (Tarski's fragment [44]) [34]. The theory is decidable using cylindrical algebraic decomposition



(CAD) [16]. The NLSAT engine implements the decision procedure for polynomial arithmetic. It uses a *model constructing satisfiability engine* (MC-SAT) to guide uses of CAD to a focused set of conflict clauses. Thus, the CAD algorithm is used on fragments of the entire formula for determining satisfiability, as opposed to globally on an entire formula.

The NLSAT engine is available as a one-shot engine for non-linear arithmetic formulas. It is also integrated with decision procedures in the CDCL(T) engine as an *end-game* solver.

2.3. SPACER

Formulas that range over *Constrained Horn Clauses* (CHC) [7, 27] are solved using a dedicated engine, SPACER, [28] that builds on an engine inspired by IC3, [6, 29]. A Constrained Horn Clause is a formula of the form

$$\forall \vec{x} . \varphi \wedge P(t) \wedge Q(s) \implies R(u)$$

or $\forall \vec{x} . \varphi \wedge P(t) \wedge Q(s) \implies \text{false}$

where φ is a formula that is interpreted over a combination of theories, P, Q, R are uninterpreted predicates, and t, s, u are terms over the bound variables $\vec{x}$ and the theories from φ . The SPACER engine searches for a model for P, Q, R such that all clauses are true under the model.

2.4. QSAT

The QSAT algorithm determines satisfiability of quantified formulas using theories that allow quantifier projection. There are two instances of QSAT, one for combinations of linear arithmetic, algebraic data-types, bit-vectors and arrays, the other is for non-linear polynomial arithmetic formulas over reals. They build on results developed in [3, 4, 41] and for solving NLSAT and Model-based projection introduced for SPACER. The algorithm is described in [Programming Z3](#).

3. Decision Procedures

We will apply the following taxonomy when discussing the theory solvers. It extends the reduction approach to decision procedures [35] as explained in the context of Z3 in [17].

- Base theories - other theory solvers build on top of base theories
 - The base theories in Z3 are Boolean theories, the theory of uninterpreted functions and the theory of arithmetic.
- Reducible theories
 - Theories that can be solved by reducing directly into base theories.
 - Instances: Arrays, IEEE floating points, algebraic data-types, recursive functions.
- Hybrid theories
 - Theories that combine non-disjoint signatures from different theories.
 - Instances: Sequences, Model-based quantifier instantiation.
- External theories
 - Theories that may be developed externally by propagating consequences and identifying conflicts.
 - Instance: an external propagator plugin.

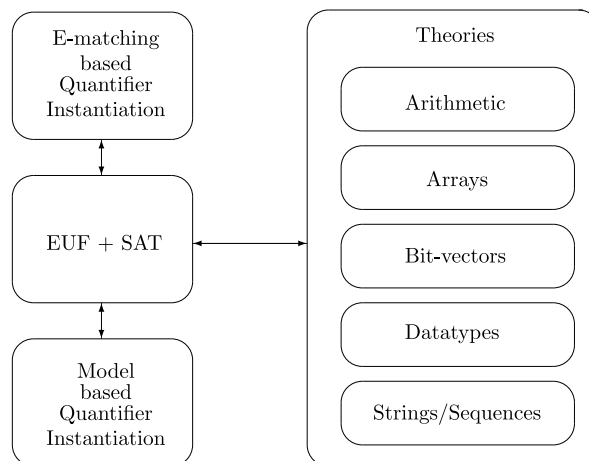


Figure 2. Architecture of Z3's SMT Core solver.

3.1. Boolean Theories

Bit-vectors are in the current solver treated as tuples of Boolean variables and all bit-vector operations are translated to Boolean SAT. The approach is called bit-blasting. Only mild equational reasoning is performed over bit-vectors. The benefits of the CDCL SAT engine have

been mostly enjoyed for applications in symbolic execution of 32-bit arithmetic instructions. Bit-blasting has its limitations: applications that use quantifiers and applications around smart contracts that use 256 bits per word are stressing this approach. A revised bit-vector solver that integrates algebraic reasoning and lazy bit-blasting is therefore currently being developed.

Cardinality and Pseudo-Boolean inequalities can also be translated into CNF clauses. It is often an advantage when there are very few variables in the summations, but the overhead of translation can quickly become impractical. Z3 therefore contains dedicated solvers for cardinality and Pseudo-Boolean constraints.

Example 2.

Bit-blasting the arithmetic expression $x + y \geq 2$ assuming both x and y consist of 2 bits results in the formula

$$\begin{aligned} z_1 &\equiv (x_1 \vee y_1) \wedge (\neg x_1 \vee \neg y_1) \\ c_2 &\equiv (x_1 \wedge y_1) \\ z_2 &\equiv (x_2 \vee y_2 \vee c_2) \wedge (\neg x_2 \vee \neg y_2 \vee \neg c_2) \wedge (x_2 \vee \neg y_2 \vee \neg c_2) \wedge (\neg x_2 \vee \neg y_2 \vee \neg c_2) \\ z_2 \end{aligned}$$

the result of the summation is given by $z = (z_1, z_2)$. c_2 is the carry of adding the first bits of x and y . Pseudo-Boolean constraints like $x + \neg y + 2z \geq 2$ can be translated in multiple ways like translating it to bit-vector addition/multiplication as before, translating it to decision-diagrams or sorting-networks.

A decision diagram for this constraint looks like

```
if z :
    true
else :
    if x :
        if ¬y :
            true
        else :
            false
    else :
        false
```

which can be translated to a propositional formula. The sorting network approach sorts the bit-sequence $\langle x, \neg y, z, z \rangle$ to a sequence $\langle s_1, s_2, s_3, s_4 \rangle$ and asserts that s_3 has to be true.

The native solver instead tracks the partial sum of already assigned booleans and sets some of the remaining ones in case it is not possible to exceed the desired bound without them.

3.2. Equality and Uninterpreted Functions

This theory captures a shared property of all theory: that equality is a congruence relation. The theory is described in many places, including in [8].

Let us use a simple example to illustrate the scope and main functionality of congruence closure based decision procedures for equality. We are given two equalities and one disequality

$$a \simeq f(f(a)), a \simeq f(f(f(a))), a \not\simeq f(a)$$

Their conjunction is unsatisfiable. Unsatisfiability is established by combining union-find [UnionFind] data-structures to maintain equivalence classes and a data-structure of E-graphs to enforce congruences. In a first step, the terms are represented by unique nodes in a DAG structure for every sub-term. We represent the DAG structure as a sequence of definitions. Then equalities and disequalities are between nodes in DAG.

$$n_1 \equiv f(a), n_2 \equiv f(n_1), n_3 \equiv f(n_2)$$

$$a \simeq n_2, a \simeq n_3, a \not\simeq n_1$$

Process equality atoms using union-find. It establishes the equivalence classes

$$\{a, n_2, n_3\}, \{n_1\}$$

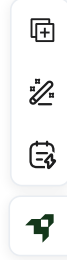
Then congruence closure triggers whenever there are two nodes, n_1, n_2 from different equivalence classes, labeled by the same function f , with equal children. In our case a, n_2 belong to the same equivalence class so the nodes n_1, n_3 are merged.

$$\{a, n_2, n_3, n_1\}$$

When the children of the equality term $a \simeq n_1$ are merged into the same equivalence class, the term $a \simeq n_1$ is set to true. This contradicts that $a \simeq n_1$ is already set to false.

3.2.1. E-Node

The E-Node data-structure [21, 23, 38] is used to implement congruence closure efficiently. The main fields of an E-Node are



$n : \langle$
 $f : \text{Func}$ function symbol
 $ts : N^*$ arguments
 $find : N \times N \times \mathcal{N}$ link to representative, sibling and class size
 $P : N^*$ list of parents
 $cg : N$ congruence representative
 $j : \text{null} \mid \text{Just} \times N$ pointer to justification and node
 $\rangle$

When a term $f(ts)$ occurs in a formula used by the solver it is compiled into an E-node n . The E-node n is initialized to

$$n \leftarrow \langle f = f, ts = ts, find = (n, n, 1), P = \text{null}, cg = n, j = \text{null} \rangle.$$

Example 3.

The result of internalizing the following term

$$f(g(a), g(a), a)$$

is

$n_1 := \langle f = a, ts = [], find = (n_1, n_1, 1), P = [n_2, n_3], cg = n_1, j = \text{null} \rangle$
 $n_2 := \langle f = g, ts = [n_1], find = (n_2, n_2, 1), P = [n_3], cg = n_2, j = \text{null} \rangle$
 $n_3 := \langle f = f, ts = [n_2, n_2, n_1], find = (n_3, n_3, 1), P = [], cg = n_3, j = \text{null} \rangle$

The field *find* is used to implement union-find. We use three fields to represent the union-find data-structure, comprising of:

- *root* - the root of the equivalence class.
- *next* - a next sibling in the equivalence class of a node. All nodes within the same equivalence class are linked in a singly-linked list.
- *sz* - the class size (number of siblings in the cyclic list). The class size is used to guide which node is made root when merging two classes.

Z3 performs eager path compression. It has the effect that the root node can always be accessed by a single pointer dereference. The singly-linked cyclic list of siblings of an equivalence class is used by operations that need to iterate over equivalence classes. Access to the equivalence class is used by E-matching and for propagation

Besides the E-nodes, the solver maintains a hash-table, we call *etable*, that is used to find the congruence root of a function application. It maps a function symbol f and list of arguments to f that are represented by root E-nodes into a congruence closure root. For the congruence closure root n it maintains the invariant $n.cg = etable[n.f, root(n.ts)]$. We write $etable[n.f, root(n.ts)]$ to indicate that the *etable* tracks function applications to root nodes. In reality, the *etable* just stores the node n and uses the *root* function when computing the current hash of n and equalities with other nodes.

3.2.2. Merge

The main functionality of congruence closure is to ensure that all equivalence classes that follow from equalities are inferred. It exposes the main function $\text{merge}(n_1, n_2, j)$ to establish that two nodes n_1, n_2 are equal under justification j . We describe justifications later in more detail. The main steps of *merge* are outlined below.

Roots $r_1 \leftarrow \text{root}(n_1), r_2 \leftarrow \text{root}(n_2)$
 assume $r_1 \neq r_2$
 assume $r_1.sz \leq r_2.sz$
Erase **for each** $p \in r_1.P$ **where** $p.cg = p$:
 erase $etable[p.f, root(p.ts)]$
Update Root $r_1.find \leftarrow r_2$
Justify justify(r_1, r_2, j)
Insert **for each** $p \in r_1.P$:
 if $etable[p.f, root(p.ts)] = \text{null}$ **then**
 $etable[p.f, root(p.ts)] \leftarrow p$
 $p.cg \leftarrow etable[p.f, root(p.ts)]$
 if $p.cg = p$ **then**
 append p to $r_2.P$
 else
 add $\langle p.cg, p, cc \rangle$ to *tomerge*

- **Roots:** congruence closure always merges current roots of equivalence classes. If the roots are equal, there is nothing to merge. We assume that r_2 is chosen as root. The main mechanism for ensuring congruence closure has sub-quadratic running time is obtained by applying Hopcroft's method of merging the *lesser* half. Z3 makes an exception to this rule if r_1 is labeled by a term that qualifies as a *value*. To quickly identify if a congruence class contains a value, such as 1, 2, $\text{cons}(3, \text{nil})$, it sets the root of a class to be the node representing the value.
- **Erase:** Entries of the *etable* that are stale are removed. An entry is stale if it points to the old root as congruence closure representative.

- **Update Root:** The root is updated along with size of equivalence classes and links into the cyclic list of siblings.
- **Justify:** The *justification* for a merged equality is updated. We describe this functionality later.
- **Insert:** Finally, the *etable* is updated by updating the *cg* field of parents from r_1 . It maintains a list *tomerge* of new nodes that are discovered congruent. An outer loop invokes *merge* on the list of new nodes until it is empty.

3.2.3. Unmerge

All operations on the E-Graph can be inverted. For this purpose, every merge is recorded in an *undo trail*. Other operations that update nodes are recorded in the same trail. For example, when p is appended to $r_2.P$ the insertion into the cyclic list of parents is recorded so that it can be reverted. Reverting a merge requires updating the *etable* and is sensitive to whether a node was a congruence root:

The effect of $\text{unmerge}(r_1, r_2)$ is as follows:

```

Erase      for each  $p \in r_2.P$  added from  $r_1.P$ :
            erase  $etable[p.f, root(p.ts)]$ 
UnJustify  unjustify( $r_1, r_2, j$ )
Revert Root  $r_1.find \leftarrow r_1$ 
Insert     for each  $p \in r_1.P$ :
            if  $p.cg \neq p \vee root(p.ts) \neq root(p.cg)$ 
               $etable[p.f, root(p.ts)] \leftarrow p.cg \leftarrow p$ 

```

3.2.4. Justifications

A justification is a reason for merging two nodes. There are two possible reasons for merging nodes:

1. A literal $\ell : s \simeq t$ is asserted. The justification is the literal ℓ .
2. Nodes are merged due to congruence closure.

$Just ::= \ell : s \simeq t \mid cc : f(ts) \simeq f(ts')$

NB: $cc : f(ts) \simeq f(ts')$ is justified *recursively* by justifying $ts \simeq ts'$.

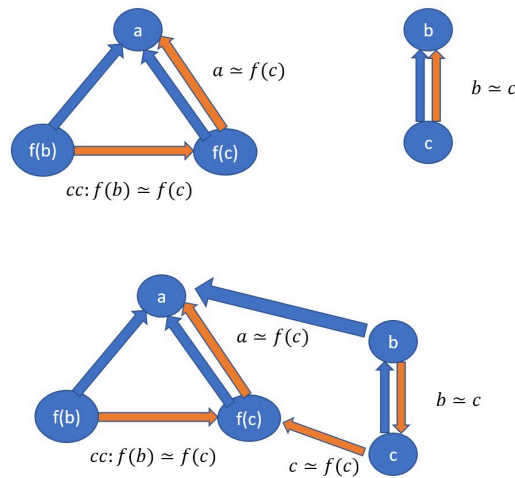
Invariant: Every non-root node points to a linked list of justifications leading to the root

NB The linked list does not follow direction of union-find.

```

 $r_1 \leftarrow root(n_1)$ 
 $r_2 \leftarrow root(n_2)$ 
 $r_1.find \leftarrow r_2$ 
old justification :  $n_1 \xrightarrow{j_1} n_1^1 \xrightarrow{j_2} n_1^2 \dots \xrightarrow{j_m} r_1$ 
new justification :  $n_1 \xrightarrow{j_1} n_1^1 \xrightarrow{j_2} n_1^2 \dots \xrightarrow{j_m} r_1$ 
add justification :  $n_1 \xrightarrow{j} n_2$ 

```



Note that not all possible justifications are tracked, if $\text{merge}(n_1, n_2, j)$ is invoked but already $\text{root}(n_1) = \text{root}(n_2)$, then the justification j for the equality of n_1, n_2 is dropped. In contrast, **egg** [25] keeps track of potential extra paths to find *short* proofs. Use cases within CDCL(T), that leverages amortized effect of backtracking search typically hedge on that the cost of finding a

strongest conflict up front is outweighed by multiple attempts that converge on sufficiently strong conflicts.

3.2.5. From justifications to proofs

We can create proofs from first-principles by using justifications. Suppose $\text{root}(s) = \text{root}(t)$ follows from a sequence $\text{merge}(s_1, t_1, \ell_1), \text{merge}(s_2, t_2, \ell_2), \dots, \text{merge}(s_k, t_k, \ell_k)$.

Then a proof of $s \simeq t$ can be extracted using overloaded functions π :

$$\begin{aligned} \pi(s \simeq t) &= \frac{\pi(s \xrightarrow{j} \dots a) \quad \frac{\pi(t \xrightarrow{j'} \dots a) \text{ symm}}{a \simeq t} \text{ trans}}{s \simeq t} \\ &\quad a \text{ is a least common } \rightarrow \text{ ancestor of } s, t \\ \pi(s \xrightarrow{j} t \xrightarrow{j'} \dots u) &= \frac{\pi(j, s \simeq t) \quad \pi(t \xrightarrow{j'} \dots u) \text{ trans}}{s \simeq u} \\ \pi(s) &= \frac{}{s \simeq s} \text{ refl} \\ \pi(\ell : s \simeq t, s \simeq t) &= \ell \\ \pi(\ell : t \simeq s, s \simeq t) &= \frac{\ell}{s \simeq t} \text{ symm} \\ \pi(cc : f(ts) \simeq f(ts'), f..) &= \frac{\pi(ts_1 \simeq ts'_1), \dots, \pi(ts_k \simeq ts'_k)}{f(ts) \simeq f(ts')} \text{ cong} \end{aligned}$$

3.2.6. Integration with Boolean reasoning

In the setting of CDCL(T), congruence closure is integrated with Boolean reasoning. For example, if $p(a)$ is assigned true, $a \simeq b$, then the Boolean predicate $p(b)$ can be propagated to true. Thus, congruence closure can be beneficial in propagation. On the other hand, propagations made by the CDCL(T) core need not be replicated by congruence closure: if the literal corresponding to $a \wedge b$ is assigned to true, then clausification of $a \wedge b$ introduces clauses $\neg(a \wedge b) \vee a$ and $\neg(a \wedge b) \vee b$, the ensure propagation to a, b . Z3 therefore dampens the role of congruence closure for Boolean connectives that are handled by CDCL.

To track Boolean assignments the E-node data-structure includes the fields

- $\langle \dots, \text{boolVar} : \mathcal{N}, \text{value} : \{\text{true}, \text{false}, \text{undef}\} \rangle$.

Furthermore, equality is treated as a special function symbol. If the children n_1, n_2 of $n_3 : n_1 \simeq n_2$ are merged, then the *value* of n_3 is set to *true*.

The E-graph also dispatches equality and disequality propagation between theories.

3.3. Arithmetic

There are several alternate engines for arithmetical constraints in Z3. Some of the engines are engineered for fragments of arithmetic, such as difference arithmetic, where all inequalities are of the form $x - y \leq k$, for k a constant, and unit-two-variable-per-inequality (UTVPI), where all inequalities are of the form $\pm x \pm y \leq k$. A new main solver for general arithmetic formulas has emerged recently, with the longer term objective of entirely replacing Z3's legacy arithmetic solver. We will here describe internals of the newer solver in more detail.

In overview, the arithmetic solver uses a waterfall model for solving arithmetic constraints.

- First, it establishes feasibility with respect to linear inequalities. Variables are solved over the rationals.
- Second, it establishes feasibility with respect to mixed integer linear constraints. Integer variables are solved if they are assigned integer values.
- Finally, it establishes feasibility with respect to non-linear polynomial constraints.

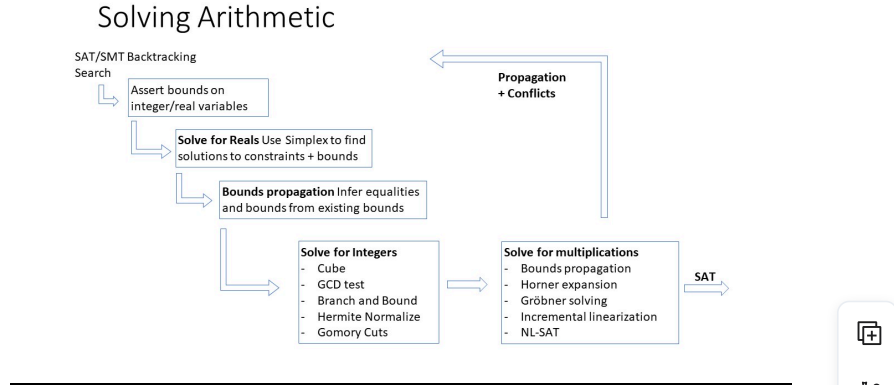


Figure 3. An overview of Arithmetical Decision Procedures in Z3

3.3.1. Rational linear arithmetic

The solver for rational linear inequalities uses a dual simplex solver, as explained in [24]. It maintains a global set of equalities of the form $A\vec{x} = 0$, and each variable x_j is assigned lower and upper bounds during search. The solver then checks for feasibility of the resulting system $A\vec{x} = 0, lo_j \leq x_j \leq hi_j, \forall j$ for dynamically changing bounds lo_j, hi_j . The bounds are justified by assignments in M .

Example 4.

For the following formula

```
x, y = Reals('x y')
solve([x >= 0, Or(x + y <= 2, x + 2*y >= 6),
       Or(x + y >= 2, x + 2*y > 4)])
```

Z3 introduces auxiliary variables s_1, s_2 and represents the formula as

$$s_1 \equiv x + y, s_2 \equiv x + 2y, \\ x \geq 0, (s_1 \leq 2 \vee s_2 \geq 6), (s_1 \geq 2 \vee s_2 > 4)$$

Only bounds (e.g., $s_1 \leq 2$) are asserted during search.

The first two equalities form the tableau. Thus, the definitions $s_1 \equiv x + y, s_2 \equiv x + 2y$ produce the equalities

$$s_1 = x + y, s_2 = x + 2y$$

They are equivalent to the normal form:

$$s_1 - x - y = 0, s_2 - x - 2y = 0$$

where s_1, s_2 are basic (dependent) and x, y are non-basic. In dual Simplex tableaux, values of a non-basic variable x_j can be chosen between lo_j and hi_j . The value of a basic variable is a function of non-basic variable values. It is the unique value that satisfies the unique row where the basic variable occurs. Pivoting swaps basic and non-basic variables and is used to get values of basic variables within bounds. For example, assume we start with a set of initial values $x = y = s_1 = s_2 = 0$ and bounds $x \geq 0, s_1 \leq 2, s_1 \geq 2$. Then s_1 has to be 2 and it is made non-basic. Instead, y becomes basic:

$$y + x - s_1 = 0, s_2 + x - 2s_1 = 0$$

The new tableau updates the assignment of variables to $x = 0, s_1 = 2, s_2 = 4, y = 2$. The resulting assignment is a model for the original formula.

Finding equal variables - cheaply

Example 5.

From equalities $x + 1 = y, y - 1 = z$ infer that $x = z$. Based on the tableau form, the solver is presented with the original equality atoms via slack variables

$$s_1 = x + u - y, s_2 = y - u - z, 1 \leq u \leq 1$$

The tableau can be solved by setting $x = 0, y = 0, z = 0, s_1 = 1, s_2 = -1, u = 1$. The slack variables are bounded when the equalities are asserted

$$s_1 = x + u - y, s_2 = y - u - z, 1 \leq u \leq 1, 0 \leq s_1 \leq 0, 0 \leq s_2 \leq 0$$

The original solution is no longer valid, the values for s_1, s_2 are out of bounds. Pivoting re-establishes feasibility using a different solution, for example

$$x = z - u - s_1, y = z - u - s_2, 1 \leq u \leq 1, 0 \leq s_1 \leq 0, 0 \leq s_2 \leq 0$$

with assignment $z = 0, x = y = -1$. The variables x and y have the same value, but must they be equal under all assignments? We can establish this directly by subtracting the right-hand sides $z - u - s_1$ and $z - u - s_2$ from another and by factoring in the constant bounds to

obtain the result 0. But subtraction is generally expensive if there are many bounded constants in the rows. The arithmetical operations are not required to infer that $x = y$. Z3 uses two facts to infer the equality

- The variables x, y have the same value in the current assignment.
- They are connected through a set of *binary* rows. That is rows, that have two unfixed variables with coefficients 1 and -1, other variables in the rows are fixed.

Variables that satisfy these two criteria must be equal.

3.3.2. Integer linear arithmetic

The mixed integer linear solver consists of several layers.

GCD consistency

Each row is first normalized by multiplying it with the least common multiple of the denominators of each coefficient. For each row, it assembles a value from the fixed variables. A variable x_j is fixed if the current values lo_j, hi_j are equal. Then it checks that the gcd of the coefficients to variables divide the fixed value. If they don't the row has no integer solution.

Example 6.

Assume we are given a row $5/6x + 3/6y + z + 5/6u = 0$, where x, y are fixed at $2 \leq x \leq 2$, $-1 \leq u \leq -1$, and z is the base variable. Then it follows that $5 + 3(y + 2z) = 0$ which has no solution over the integers: The greatest common divisor of coefficients to the non-fixed variables (3) does not divide the constant offset from the fixed variables (5).

This GCD test is extended to as follows. For each row $ax + by + c = 0$, where

- x, y, a, b, c are vectors of constants and variables, respectively.
- the coefficients in a are all the same and smaller than the coefficients in b
- the variables x are bounded

Let $l := a \cdot lb(x), u := a \cdot ub(x)$. That is, the lower and upper bounds for $a \cdot x$ based on the bounds for x .

Let $ll := \lceil \frac{l}{\gcd(b,c)} \rceil, uu := \lfloor \frac{u}{\gcd(b,c)} \rfloor$. If $uu > ll$, there is no solution for x within the bounds for x .

Patching

When a tableau with integer variables is feasible, all non-basic variables can be assumed to be integers: Dual simplex moves non-basic variables to their bounds and the bounds are integers. Only the basic variables could be non-integral (they are still within their bounds). Patching seeks changing values to non-basic values to move as many basic variables to integers.

Example 7.

Suppose we are given a tableau of the form

$$\begin{aligned} y - \frac{1}{2}x &= 0 \\ z - \frac{1}{3}x &= 0 \end{aligned}$$

where y, z are basic variables and x has bounds $[3, 10]$, y has bounds $[-3, 4]$, z has bounds $[-5, 12]$. The variable x is initially assigned at the bound $\beta(x) = 3$. Then $\beta(y) = \frac{3}{2}$ and $\beta(z) = 1$. But neither y nor z is close to their bounds. We can move x to 8 without violating the bound for y because of $y - \frac{1}{2}x = 0$. Thus, the freedom interval for x is the range $[3, 8]$ and within this range there is a solution, $x = 6$, where y, z are integers and within their bounds.

Z3 patching works by shifting a non-basic variable by δ to make a dependent basic variable integral.

```
for  $x_b$  in basic variables, such that  $\beta(x_b) \notin \mathbb{Z}$ :
  for  $\alpha y$  in row of  $x_b$  with  $\alpha \notin \mathbb{Z}$ :
    find  $\delta$  such that the update  $\beta(y) := \beta(y) + \delta$ 
    is feasible and fixes more basic variables than it breaks
```

Given a row $x_b + \alpha y + r = 0$, where x_b is a basic variable, y is non-basic variable multiplied by the fraction α , and r is the remaining of the row, the shift amount δ is computed based on the following analysis.

Take first the fractional parts of $\beta(x_b)$ and α :

- $f_x := x_1/x_2 := \text{frac}(\beta(x_b))$, s.t. $0 < x_1 < x_2$, and x_1, x_2 are mutually prime.
- $f_\alpha := a_1/a_2 := \text{frac}(\alpha)$, s.t. $0 < a_1 < a_2$, and a_1, a_2 are mutually prime.

The goal is to compute two values for δ , one negative, one positive, of minimal absolute values, such that $\beta(x_b) + \delta \in \mathbb{Z}$, where $\mathbb{Z}$ is the set of integers. Rewriting, we have $L := \frac{x_1}{x_2} + \frac{a_1}{a_2}\delta \in \mathbb{Z}$.

If $L \in \mathbb{Z}$ then $a_2 \frac{x_1}{x_2} + a_1\delta$ is also an integer. Therefore $a_2 \frac{x_1}{x_2} \in \mathbb{Z}$. That means $a_2 := tx_2$ for some $t \in \mathbb{Z}$, because x_1 and x_2 are coprime. By substituting a_2 with x_2t we get



$L := \frac{x_1}{x_2} + \frac{a_1}{x_2 t} \delta$ and $Lx_2 := x_1 + \frac{a_1}{t} \delta \in Z$. Since $t \uparrow a_2$, and a_2 and a_1 are coprime, $t \uparrow \delta$. Therefore, we search for δ in form $\delta := mt$, $m \in Z$. We obtain $L := \frac{x_1}{x_2} + \frac{a_1}{x_2 t} mt = \frac{x_1 + ma_1}{x_2}$. From $L \in Z$ follows $x_2 k = x_1 + ma_1$ for some $k \in Z$. We can rewrite the last equality as $x_1 = a_1 m - x_2 k$. Because $x_2 \uparrow a_2$, and a_1 and a_2 are coprime, x_2 and a_1 are mutually prime too. That means that for some u, v we have $1 = a_1 u + x_2 v$. We can show that if $\delta := -utx_1$ then $\frac{x_1}{x_2} + \frac{a_1}{a_2} \delta \in Z$.

From the other side, for any $\gamma \in Z$ satisfying $\frac{x_1}{x_2} + \frac{a_1}{a_2} \gamma \in Z$ holds $\frac{a_1(\delta - \gamma)}{a_2} \in Z$. Since a_1 and a_2 are coprime, $a_2 \uparrow (\delta - \gamma)$, and $\gamma := \delta \pmod{a_2}$. We conclude that $\delta^+ = \delta \pmod{a_2}$, and $\delta^- = \delta^+ - a_2$.

Cubes

An important factor in solving more satisfiable integer arithmetic instances is a method by Bromberger and Weidenbach [12, 13]. It allows detecting feasible inequalities over integer variables by solving a stronger linear system. In addition, we observed that the default strengthening proposed by Bromberger and Weidenbach can often be avoided: integer solutions can be guaranteed from weaker systems.

We will here recall the main method and our twist. In the following we let A, A' range over integer matrices and a, b, c over integer vectors. The 1-norm $\|A\|_1$ of a matrix is a column vector, such that each entry i is the sum of the absolute values of the elements in the corresponding row A_i . We write $\|A_i\|_1$ to directly access the 1-norm of a row i .

A (unit) *cube* is a polyhedron that is a Cartesian product of intervals of length one for each variable. Since each variable therefore contains an integer point, the interior of the polyhedron contains an integer point. The condition for a convex polyhedron to contain a cube can be recast as follows:

Example 8.

Suppose we have $3x + y \leq 9 \wedge -3y \leq -2$ and wish to find an integer solution. By solving $3x + y \leq 9 - \frac{1}{2}(3 + 1) = 7, -3y \leq -2 - \frac{1}{2}3 = -3.5$ we find a model where $y = \frac{7}{6}, x = 0$. After rounding y to 1 and maintaining x at 0 we obtain an integer solution to the original inequalities.

Our twist on Bromberger and Weidenbach's method is to avoid strengthening on selected inequalities. First, we note that *difference* inequalities of the form $x - y \leq k$, where x, y are integer variables and k is an integer offset need not be strengthened. For octagon constraints $\pm x \pm y \leq k$, there is a boundary condition: they need only require strengthening if x, y are assigned at mid-points between integral solutions. For example, if $V(x) = \frac{1}{2}$ and $V(y) = \frac{3}{2}$, for $x + y \leq 2$. Our approach is described in detail in [5].

Branching

Similar to traditional MIP branch-and-bound methods, the solver creates somewhat eagerly case splits on bounds of integer variables if the dual simplex solver fails to assign them integer values. For example, Simplex may assign an integer variable x , the value $\frac{1}{2}$, in which case z3 creates a literal $x \leq 0$ that triggers two branches $x \leq 0$ and $\neg(x \leq 0) \equiv x \geq 1$.

Gomory and Hermite cuts

The arithmetic solver produces Gomory cuts from rows where the basic variables are non-integers after the non-basic variables have been pushed to the bounds. It also incorporates algorithms from [14, 22] to generate cuts after the linear systems have been transformed into Hermite matrices.

3.3.3. Non-linear arithmetic

Similar to solving for integer feasibility, the arithmetic solver solves constraints over polynomials using a waterfall model for non-linear constraints. At the basis it maintains for every monomial term $x \cdot x \cdot y$ an equation $m = x \cdot x \cdot y$, where m is a variable that represents the monomial $x \cdot x \cdot y$. The module for non-linear arithmetic then attempts to establish a valuation V where $V(m) = V(x) \cdot V(x) \cdot V(y)$, or derive a consequence that no such valuation exists. The stages in the waterfall model are summarized in the following paragraphs.

Bounds propagation on monomials

A relatively inexpensive step is to propagate and check bounds based on non-linear constraints. For example, for $y \geq 3$, then $m = x \cdot x \cdot y \geq 3$, if furthermore $x \leq -2$, we have the strengthened bound $m \geq 12$. Bounds propagation can also flow from bounds on m to bounds on the variables that make up the monomial, such that when $m \geq 8, 1 \leq y \leq 2, x \leq 0$, then we learn the stronger bound $x \leq -2$ on x .

Bounds propagation with Horner expansions

If $x \geq 2, y \geq -1, z \geq 2$, then $y + z \geq 1$ and therefore $x \cdot (y + z) \geq 2$, but we would not be able to deduce this fact if combining bounds individually for $x \cdot y$ and $x \cdot z$ because no bounds can be inferred for $x \cdot y$ in isolation. The solver therefore attempts different re-distribution of multiplication in an effort to find stronger bounds.

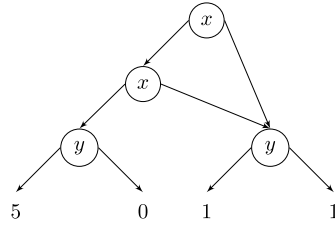
Gröbner reduction



Many constraints using polynomials can be checked for consistency by normalization. The Gröbner basis reduction algorithms provide a means to check consistency of a set of polynomial equations (which is complete if adding the Nullstellensatz checks). Z3 uses a best effort Gröbner basis reduction to find inconsistencies, cheaply, and propagate consequences. It first extracts equalities from Simplex rows where the basic variable is bounded. It adds equations $v = v_1 \dots v_k$ for arithmetic variables that are defined as monomials.

- Equations are pre-solved if they are linear and can be split into two groups, one containing a single variable that has a lower (upper) bound, the other with more than two variables with upper (lower) bounds. This avoids losing bounds information during completion.
- After (partial) completion, perform constant propagation for equalities of the form $x = 0$. The equality is propagated to the linear solver.
- After (partial) completion, perform factorization for factors of the form $xyp \simeq 0$ where x, y are variables and p is linear. We infer the clause $xyp \simeq 0 \implies x \simeq 0 \vee y \simeq 0 \vee p \simeq 0$.

We use an adaptation of ZDD (Zero suppressed decision diagrams [37, 39]) to represent polynomials. The representation has the advantage that polynomials are stored in a shared data-structure and operations over polynomials are memorized. A polynomial over the real is represented as an acyclic graph, where nodes are labeled by variables and edges are labeled by coefficients. For example, the polynomial $5x^2y + xy + y + x + 1$ is represented by the acyclic graph shown below.



The root node labeled by x represents the polynomial $x \cdot l + r$, where l is the polynomial of the left sub-graph and r the polynomial of the right sub-graph. The left sub-graph is allowed to be labeled again by x , but the right sub-graph may only have nodes labeled by variables that are smaller in a fixed ordering. The fixed ordering used in this example sets x above y . Then the polynomial for the right sub-graph is $y + 1$, and the polynomial with the left sub-graph is $5xy + (y + 1)$.

The Gröbner module performs a set of partial completion steps, preferring to eliminate variables that can be isolated, and expanding a bounded number of super-position steps (reductions by S-polynomials).

Incremental linearization

Following [15] we incrementally linearize monomial constraints. For example, we include lemmas of the form $x = 0 \rightarrow m = 0$ and $x = 1 \rightarrow m = y$, for $m = x^2y$.

NLSat

As an end-game attempt, the solver attempts to solve the non-linear constraints using a complete solver for Tarski's fragment supported by the NLSat solver [34].

3.3.4. Data-structures

Z3 uses infinite precision integers and rationals. This ensures that all operations are sound and not subject to errors introduced by rounding. It uses a sparse matrix representation for the Dual Simplex tableau. Note that pivoting may turn an otherwise sparse matrix into a dense form. We also created a version that uses an LRU decomposition and floating point numbers but found that extending this version with efficient backtracking was not practical compared to the straight-forward sparse matrix format.

The arithmetic solver reduces operations such as integer modulus, remainder, and division into equations. It eagerly compiles bounds axioms of the form $x \geq 3 \implies x \geq 2$ for atomic formulas $x \geq 3, x \geq 2$.

3.4. Reducible Theories

3.4.1. Refinement Types

Let us illustrate a use of *reduction* from richer theories to base theories based on a simple example based on refinement types. It encodes refinement types using auxiliary functions, as explained in [30]. Refinement types are not part of Z3, so the decision procedure we outline here is not available out of the box. One way to realize this theory is externally through the User-Propagator facility.

Abstractly, a refinement type of sort S uses a predicate p over S . At least one element of S must satisfy p for the construction to make sense. The refinement type $S \mid p$ represents the elements of S that satisfy p . The properties we need to know about elements of $S \mid p$ can be encoded using two auxiliary functions that form a surjection $relax$ from $S \mid p$ into S with a partial inverse $restrict$ that maps elements from S into $S \mid p$. The properties of these functions are summarized as follows:

$$\begin{aligned} p &: S \rightarrow Bool \\ relax &: S \mid p \rightarrow S \\ restrict &: S \rightarrow S \mid p \\ \forall x : S \mid p . restrict(relax(x)) &= x \\ \forall s : S . p(s) \rightarrow relax(restrict(s)) &= s \\ \forall x : S \mid p . p(relax(x)) & \end{aligned}$$

Let us illustrate the sort of natural numbers as a refinement type of integers:

Example 9.

$$\begin{aligned} sort\ Nat &= Int \mid \lambda x . x \geq 0 \\ \forall n : Nat . restrict(relax(n)) &= n \wedge relax(n) \geq 0 \\ \forall i : Int . i \geq 0 \rightarrow relax(restrict(i)) &= i \end{aligned}$$

We obtain a theory solver for formulas with refinement types by instantiating these axioms whenever there is a term t introduced of sort $S \mid p$ introduced as part of the input or during search (from instantiating quantifiers). The main challenge with supporting this theory is to ensure that the new terms introduced from axiom instantiation are bounded. We don't want the solver to create terms $relax(restrict(relax(restrict(...))))$.

- For every sub-term of the form $restrict(t)$, where t is not $relax(t')$ instantiate the axiom:
 - $p(t) \Rightarrow relax(restrict(t)) = t$
- For every term t of sort $S \mid p$ instantiate the axioms:
 - $restrict(relax(t)) = t$
 - $p(relax(t))$

3.4.2. Arrays

The theory of arrays in SMTLIB is formally based on two functions `select` and `store` and an axiomatization

$$select(store(A, i, v), j) = \text{if } i \simeq j \text{ then } v \text{ else } select(A, j)$$

$$\forall i . select(A, i) \simeq select(B, i) \implies A \simeq B$$

for every array A, B , index i, j and value v . Alluding to the intent that the theory of arrays is useful for modeling arrays in programming languages will henceforth use $A[i]$ when we mean $select(A, i)$.

Z3 reduces the theory of arrays to reasoning about uninterpreted functions. It furthermore treats arrays as function spaces, which assuming more axioms about arrays. The first-order theory of arrays axiomatized above enjoys compactness and so the following formula is satisfiable²

$$\forall a : Array(Int, Int) . \exists k . \forall i \geq k . a[i] \simeq 0.$$

The same formula is not satisfiable when arrays range over function spaces. The distinction is only relevant when checking satisfiability of quantified formulas over arrays. In addition to functions `store` and `select` Z3 includes built-in functions `map`, `const` and the operator `as-array` and lambdas. It includes also the function `ext` that provides a witness term for distinct arrays, and $\delta(A)$ for accessing a *default* value.

The central functionality of the decision procedure for arrays is to ensure that a satisfying model under the theory of EUF translates to a satisfying model in the theory of arrays. To this end, the main service of the theory solver is to saturate the search state with β reduction axioms for array terms that admit beta-reduction. We call these terms λ terms and they are defined by the beta-reduction axioms:

$$\begin{aligned} \beta(store(A, j, v)[i]) &= \text{if } i \simeq j \text{ then } v \text{ else } A[i] \\ \beta(map(f, A, B)[i]) &= f(A[i], B[i]) \\ \beta(as-array(f)[i]) &= f(i) \\ \beta(const(v)[i]) &= v \\ \beta((\lambda x . M)[i]) &= M[i/x] \end{aligned}$$

The reduction into EUF, is then, in a nutshell, an application of the following inference rule:

$$\frac{b \text{ is a lambda term} \quad a[j] \text{ is a term} \quad b \sim a \text{ (} a, b \text{ are equal under EUF)}}{b[j] = \beta(b[j])}$$

together with an inference rule to enforce extensionality



$$A[\text{ext}(A, B)] \simeq B[\text{ext}(A, B)] \implies A \simeq B$$

The decision procedure enforces several throttles [17] to avoid instantiating axioms when they are redundant. We here describe the main data-structures used by the instantiation engine.

The decision procedure maintains for every array node n the following sets

$$\begin{aligned} \text{ParentSelects}(n) &= \{ A[i] \mid A \sim n \} \\ \text{ParentLambdas}(n) &= \{ \text{store}(A, i, v) \mid A \sim n \} \cup \{ \text{map}(f, \dots, A, \dots) \mid A \sim n \} \\ \text{Lambdas}(n) &= \{ \text{const}(v) \mid \text{const}(v) \sim n \} \cup \\ &\quad \{ \text{map}(f, \dots) \mid \text{map}(f, \dots) \sim n \} \cup \\ &\quad \{ \text{store}(\dots) \mid \text{store}(\dots) \sim n \} \cup \\ &\quad \{ \text{as-array}(f) \mid \text{as-array}(f) \sim n \} \cup \\ &\quad \{ \lambda x. M \mid \lambda x. M \sim n \} \end{aligned}$$

When n_1 is merged with n_2 , and n_1 is the new root, the sets from n_2 are added to n_1 . The merge also looks for new redexes when n and n' are merged:

$$\frac{n \sim n' \quad A[j] \in \text{ParentSelects}(n) \quad \lambda \in \text{Lambdas}(n') \cup \text{ParentLambdas}(n')}{n \simeq n' \implies \lambda[j] = \beta(\lambda[j])}$$

For enforcing $\text{store}(A, i, v)[j] = \beta(\text{store}(A, i, v)[j])$, the solver only instantiates the axiom $i \simeq j \vee \text{store}(A, i, v)[j] \simeq A[j]$ because the axiom $\text{store}(A, i, v)[i] \simeq v$ is instantiated eagerly when the term $\text{store}(A, i, v)$ is created.

Extensionality is enforced on pairs of array terms A, B , that are in *shared* positions. An array term is in a shared position when it occurs on one side of an equality or under a function that is not part of the theory of arrays. Furthermore, to ensure congruence for lambdas, if $A \equiv \lambda x. M$ is a lambda term in a shared position and B is in a shared position, the axiom $\lambda x. M \simeq B \implies \forall i. M[x/i] \simeq B[i]$ is instantiated. Thus, lambda equality is at the time of writing enforced by a reduction to quantified formulas.

3.4.3. Algebraic Datatypes

The theory of algebraic data-types is an SMTLIB2 standard [2]. The theory allows declaring a datatype sort, or a set of mutually recursive datatypes sorts. The elements of an algebraic datatype are the least set generated by the constructors in the type declaration. You can declare algebraic data-types in SMTLIB2 using the `declare-datatypes` command

```
(declare-datatypes ((Tree 1) (TreeList 1))
  (
    (par (T) (Tree leaf (node (value T) (children (TreeList T))))
      (par (T) (TreeList nil (cons (car (Tree T)) (cdr (TreeList T))))))
  )
)
```

A legacy format that is less flexible, but a bit easier to formulate, is also available.

```
(declare-datatypes (T) ((Tree leaf (node (value T) (children TreeList)))
  (TreeList nil (cons (car Tree) (cdr TreeList)))))
```

Z3 supports datatypes nested with sequences and arrays. The example below uses `'Stmt'` nested in a sequence.

```
(declare-sort Expr)
(declare-sort Var)
(declare-datatypes ((Stmt 0))
  (((Assignment (lval Var) (rval Expr))
    (If (cond Expr) (th Stmt) (el Stmt))
    (Seq (stmts (Seq Stmt)))))
```

For arrays, datatypes are allowed only in the range of arrays.

Provide example with nested arrays

We will in the following use the notation C for a constructor, for example `Assignment` is a constructor, $\text{acc}_1, \dots, \text{acc}_n$ for accessors corresponding to C , for example `lval` and `rval` are accessors corresponding to the constructor `Assignment`. The predicate symbol $\text{isC}(t)$ is true if the term t is equal to a term headed by the constructor C . The construct $\{t \text{ with } \text{field} := s\}$ assigns the term s to an accessor field for the term t . Note that it is possible to apply an accessor to a data-type constructor that does not match. For example, taking the head of an empty list, $\text{head}(\text{nil})$. The interpretation of $\text{head}(\text{nil})$ is not fixed by the theory, instead head is treated as an uninterpreted function.

The theory solver for algebraic datatypes is implemented by adding theory axioms on demand. The theory axioms rely on the theory of uninterpreted functions. It builds a finite interpretation for every node of sort data-type and adds theory axioms on demand. Every E-node n of data-type sort is assigned a constructor representation, initially *null*. If n is equated to a node n' whose function is labeled by a constructor C , the representative for n is labeled by n' .

Saturation Rules

The saturation rules add theory axioms on demand to enforce that the theory of algebraic datatypes axioms are satisfied in a consistent state. Accessor axioms, update-field axioms are

added as soon as terms are created. Recognizer axioms are added when the truth value of a recognizer is asserted. If a datatype has a single constructor, the recognizer is asserted as a unit. The occurs check is applied lazily. It ensures that the partially constructed model corresponds to acyclic algebraic datatypes.

Accessor axioms:

If n is a constructor node $C(a_1, \dots, a_m)$ assert the axioms

$$n \simeq C(a_1, \dots, a_m) \implies acc_1(n) \simeq a_1 \wedge \dots \wedge acc_m(n) \simeq a_m$$

Update field axioms:

If n is of the form $n := \{r \text{ with } field := v\}$, where $field$ is an accessor for constructor C , then assert the axioms

$$\begin{aligned} isC(r) &\implies acc_j(n) \simeq acc_j(r) \text{ for } acc_j \neq field \\ isC(r) &\implies field(n) \simeq v \\ \neg isC(r) &\implies n \simeq r \\ isC(r) &\implies isC(n) \end{aligned}$$

Recognizers

For recognizer atom we have to ensure that a node is assigned to a constructor that is consistent with the assignment to the recognizer atom. Thus, if $isC(n)$ is asserted to true, then n must be equated to a term with constructor C . Conversely, if n is assigned with a constructor $C' \neq C$, then $isC(n)$ is false. The decision procedure ensures this correspondence lazily. If $isC(n)$ is asserted to true, then it ensures the axiom

$$isC(n) \implies n \simeq C(acc_1(n), \dots, acc_m(n))$$

where $acc_1, \dots, acc_m$ are the accessors for C .

If $isC(n)$ is asserted to false, but n equal to a node n' that is headed by C , then it creates the conflict clause:

$$n \simeq n' \implies isC(n) \text{ if } n' \text{ is of the form } C(\dots)$$

Something about covering constructors so all possible constructors are tested.

Occurs check

An occurs check violation is detected when there is a state satisfying the properties $n_1 \simeq n'_1, n_2 \simeq n'_2, \dots, n_k \simeq n'_1$ where each n'_i is of the form $C_i(\dots, n_{i+1}, \dots)$. Occurs checks are performed lazily in a *final* check.

Model construction rules

Dually to saturation rules, the theory solver builds a finite model for algebraic datatypes. Model construction is interleaved with the CDCL(T) engine in a way that has subtle consequences. Model construction drives the constraints to a goal state where all terms ranging over an algebraic datatype have been assigned a constructor and all saturation rules have been applied. If there is a term that has not been assigned a constructor, the solver attempts to first guess an assignment to the term based on a non-recursive base case for the data-type (for mutually recursive data-types some types don't have non-recursive base cases, it is possible to drive towards a nested sub-term that has). The fact that the solver can guess a fixed base case constructor during model construction relies on the assumptions for theory combinations: other theories need only distinguish whether data-type terms are equal or distinct. The shape of terms is opaque to other theories. If a term t cannot be assigned a non-recursive base case (say *nil*), it is assigned a non-recursive constructor (say, *cons*), that eventually allows assigning t to a term that is arbitrarily deep and therefore can be distinct from any set of other terms $t_2, t_3, \dots$

The approach for model construction does not work if we introduce a sub-term predicate $t \preceq s$. The theory of algebraic data-types can be extended to the subterm relation while remaining decidable. If t, s range over an algebraic data-type with two base case constructors nil_1 and nil_2 , and $s = nil_2$. Then *unfairly* guessing only nil_1 before guessing *cons*, but neglecting nil_2 leads to non-termination of a model constructing decision procedure with the sub-term relation.

Quantifier Elimination and Algebraic Datatypes

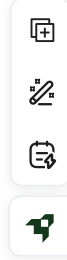
The theory of algebraic data-types admits partial quantifier elimination.

Present MBP for ADT here specifically?

3.5. Hybrid Theories

A prime example of a hybrid theory in Z3 is the theory of strings, regular expressions and sequences.

The theory of strings and regular expressions has entered mainstream SMT solving thanks to community efforts around standardization and solvers. The SMTLIB2 format for unicode strings [2]. It integrates operations that mix equational solving over the free monoid with integer arithmetic (for string lengths and extracting sub-strings). Regular expression constraints furthermore effectively introduce constraints that require unfolding recursive relations. Z3 uses symbolic derivatives [43] to handle regular expressions, noteworthy, with complementation and intersection handled by derivatives.



A second prolific example of a hybrid theory is Z3's model-based quantifier instantiation engine (MBQI). Here, a theory is encoded using a quantifier. The MBQI engine supports extensions of Bradley-Manna-Sipma's array property fragment [11] that effectively combines arithmetic with uninterpreted functions.

3.5.1. Sequences/Strings/Regex

3.5.2. Floating points

Floating point semantics can mainly be defined in terms of bit-vector operations. The solver for floating points uses this connection to reduce the theory of floating points to bit-vectors. There are also operations on floating points that connect with the theory of Reals. For formulas combining floating points and reals, the theory solver bridges bit-vector values with bounds over the Reals.

4. Data-structure and algorithm internals

This section pops the abstraction level and examines the main data-structures, organization, algorithms and heuristics that are central to implementation of Z3. We examine the data-structures used for terms and formulas and then describe the setting for how theory solvers interact with the main CDCL(T) engine.

4.1. Terms and Formulas

Z3 treats terms and formulas as a single entity, expressions. Expressions of Boolean type can be used as formulas, but they can also be nested within non-Boolean terms, as arguments to functions. Expressions are *hash-consed*, meaning they are unique up to term structure. It uses a hash-table when constructing a fresh term, such that if the same function is applied to the same arguments as a previously constructed term, the result of creating the term is the same pointer as the old term.

Variables that are to be bound by quantifiers are indexed using de-Brujin conventions, such that the index used for the innermost quantified variable is 0, and incremented outwards. For example, $\exists x \forall y, z, p(x, y, z)$ uses de-Brujin indices 0 for z and 1 for y , and 2 for x . To spare users an error-prone process of keeping track of indices, the external API offers a facility to create a quantified expression using a set of constant expressions instead of variables.

A representation of Z3 terms using the Koka language is as follows

```
type Expr
  Var{ index : int; sort : Sort }
  App{ f : FuncDecl; args : list<Expr> }
  Quantifier{ b : Binder; decl : list<Declaration>; body : Expr; ...}
```

4.2. Representing assertions

From input and during pre-processing assertions are represented in a structure of the form

```
<
   $\varphi$  : Bool          the current formula
   $\pi$  : Proof          a justification for the formula
   $\delta$  : Dependency   A dependency structure for the formula
>
```

For a standard input assertion φ the structure is initialized as $\langle \varphi, \text{asserted}(\varphi), \text{null} \rangle$. The proof of φ is that it was asserted. There are no dependencies for φ .

Input assertions can also be *tracked* for unsatisfiable cores. The SMTLIB2 contains syntax for adding *named* formulas. The formulas are tracked by their names and the solver can produce unsatisfiable cores over the tracking names. Internally, the names are treated as (fresh) Boolean literals. Thus, formulas can also be tracked by arbitrary formulas. The programmatic API contains a function `z3_solver_assert_and_track` (and variants surfaced in various programming languages) that allows asserting a formula modulo a tracking assumption. When adding tracked formulas using tracking assumption p , the structure is initialized as $\langle \varphi, \text{asserted}(\varphi), p \rangle$.

Tactics [8] perform updates on the triple representation by applying inferences. Properly producing proof objects for tactics and tracking dependencies is supported in a best-effort manner: several tactics are disabled when the user requires proof objects or tracking dependencies. Dependencies can be viewed as light-weight proof objects. The mechanism for tracking dependencies is based on tracking a cone of influence, by taking unions, of formulas used to derive a new consequence.

4.3. Compiling terms and formulas into clauses and E-nodes

While the input to Z3 is a set of formulas, the core solver works with clauses and E-nodes. The *internalization* process converts formulas into the internal data-structures bottom-up. Every sub-term that corresponds to a Boolean connective is compiled, thanks to the Tseitin transformation, into a fresh literal with associated set of clauses that define the literal. The terms from Boolean connectives may be assigned E-nodes, but the nodes do not participate in



congruence rules, nor are, by default, allowed to be merged with the constants *true* or *false*: The CDCL(T) engine already enforces congruence rules and consistency with assignments to *true/false*. The default is overridden if a term occurs as an argument to a non-Boolean connective. For example, in the term $f(a \wedge b)$, the node $n_{a \wedge b}$ merges with *true/false* when the truth value to the literal $\ell_{a \wedge b}$ is propagated. In this way, congruence closure can ensure consistency with respect to occurrences of f .

Compilation of terms into E-nodes uses a top-down traversal of terms. The top-down traversal is replicated in each theory solver; where the compilation of terms depends on the solver. The arithmetic solver, for instance, compiles the term $x + y + x$ into a node n and adds the basic row $n = 2x + y$. Top-down compilation does not play well with small static stack sizes allocated by C++ runtimes. The new core in Z3 therefore introduces a visitor pattern to allow top-down processing without using recursion stacks.

E-nodes are compiled dynamically during search. When a quantifier is instantiated or a solver introduces a new lemma that uses new terms, the E-nodes corresponding to new terms are introduced at a search level that may later be popped. Learned clauses may refer to these new terms in a scope where they are no longer internalized into E-nodes. The core solver therefore interacts with the scope mechanism of the CDCL solver to *replay* literals that are defined in deeper scopes.

4.4. Interaction with the CDCL(T) core

In Z3, the solver for the theory of equality acts as a gate-keeper between the SAT solver and solvers for other theories. As Section 2.1 outlines, the SAT solver maintains a set of ground clauses F and performs case splits and propagations and in this process consults the theory solver $Theory(M; F)$ for the next propagation, case split or whether M is a theory consistent model of F .

There is a good reason why the theory of equality is central: this theory is shared among all other theories. Disjoint theory solvers establish satisfiability by reconciling equalities among shared sub-terms.

Every E-node has an attached set of theory solvers. A theory solver for a theory T gets attached to a node n when

- $n.f$, the function symbol of n , is from the signature of T .
- $n.P$, a parent of n , belongs to T .
- The sort of n is handled by T .

Then, when a Boolean node n is assigned to *true* or *false* by the CDCL solver, the truth assignment to n is broadcast to all solvers attached to n . Similarly, when two nodes n, n' are merged, and the theory T is attached to a sibling of n and a sibling of n' , then the equality $n \simeq n'$ is broadcast to T .

In some cases, the core also broadcasts disequalities between E-nodes. Solvers cannot rely on complete propagation of disequalities, it is generally too costly to enumerate all pairwise disequalities. Propagation of disequalities is triggered by when the CDCL(T) core asserts a disequality $n \not\simeq n'$, or when a theory solver propagates a disequality.

The set of terms tracked by a given theory is always a subset of all terms used in a formula. Particularly, if a formula encodes bit-vector properties, there are thousands of terms used for encoding Boolean formulas. Those terms are irrelevant to the theories. Each theory therefore maintains a dense encoding of the terms it tracks by maintaining an array of E-nodes associated with the theory. The index into this array is the identifier of the theory variable. This identifier is stored in the E-node data-structure so that when the core broadcasts an equality about $n \simeq n'$ it accesses the theory variables v_n and $v_{n'}$ that are indices to the array and broadcast $v_n \simeq v_{n'}$. The original nodes n, n' are accessed from the theory solver's array, together with other attributes of n, n' that are specific to the theory solver. For example, the array solver maintained several attributes, such as the select terms where an array node occurs.

4.5. Theory Lemmas, Axioms and Propagations

Theory solvers generate theory lemmas, axioms and unit propagations. Theory axioms follow the following life-time contract:

- Axioms are *never* garbage collected within the current scope.
- Axioms are *always* garbage collected when the current scope is popped.
- Lemmas are *possibly* garbage collected within the current scope.
- Lemmas that survive garbage collected are re-asserted with lower scopes if they
 - contain atoms that were created above the base level, or
 - trigger a conflict or unit propagation.

Theory propagations are not replayed. The solver also behaves differently when relevancy propagation is set to 2: then theory lemmas that would otherwise correspond to binary clauses



whose atomic formulas have been internalized at the base level (so they do not have to be re-created during backtracking) are added to the watch lists directly without creating a clause. The watch list is not garbage collected so they persist.

4.6. Model-based theory combination

The main theories, arrays, arithmetic, algebraic data-types, uninterpreted functions are signature disjoint. A central ingredient in combining theory solvers into a complete solver is to reconcile equalities between shared sub-terms. Z3 uses a method called *model-based theory combination* to broadcast equalities on demand.

Example 10.

Consider the following formula that combines arithmetic for integers x, y and the uninterpreted function f .

$$x = f(y - 1), f(x) \neq f(y), 0 \leq x \leq 1, 0 \leq y \leq 1$$

The purified version of the formula takes the form, where we introduce a proxy variable z for the term $y - 1$:

$$x = f(z), f(x) \neq f(y), 0 \leq x \leq 1, 0 \leq y \leq 1, z = y - 1$$

Purification is a formal tool to help us reason about theory combinations. In practice, no new variables are introduced. The proxy variable z is just a shorthand for the term $y - 1$.

Returning to our example, the theory solver for f and the solver for arithmetic only need to reconcile equalities between shared sub-terms x, y, z in order to establish satisfiability of the overall formula. The theory of equality that handles f is agnostic to the specific values of x, y, z but needs to know which values are equal and disequal.

Example 11.

Suppose we have a model over arithmetic that sets $x = y = 1, z = 0$ and a model for f that sets $x = \star_1, y = \star_2, z = \star_1, f(x) = \text{if } x = \star_1 \text{ then } \star_1 \text{ else } \star_2$. The two models cannot be reconciled, as the equality $x \simeq y$ from the arithmetical model contradicts the partition imposed by the EUF model.

The way Z3 reconciles the equalities is by inspecting the E-graph on final checks. If a theory solver induces an equality between two nodes n, n' , but $n \not\sim n'$ (the nodes are not in the same equivalence class), then the theory solver creates a fresh atom $n \simeq n'$ and sets the *phase* of the atom to *true*. The atom is introduced to the CDCL(T) solver that performs a case split on it. The *phase* of a literal is a flag that instructs the solver to choose the case split. It first case splits on $n \simeq n'$ and only backtracking and propagation may force the case $n \not\sim n'$.

Example 12.

The arithmetic solver creates the literal $x \simeq y$ and the CDCL(T) core assumes this equality. Since x, y are shared, the equality is known to both theories. The equality contradicts the EUF theory, and it infers $x \not\sim y$. The arithmetic solver is able to produce a new model, however, where $x = 0, y = 1, z = 0$. It creates the equality atom $x \simeq z$, which does not contradict the EUF model.

Note that equalities for finite domain theories, such as bit-vectors, are determined by complete propositional assignments in final states that are visible to all theories because all terms are assigned values from the finite domain.

4.7. Relevancy filtering

The E-matching and MBQI methods for instantiating quantifiers, and the methods for solving the array theory, rely on essentially checking constraints that a satisfying model must meet. The number of quantifier instantiations and axiom instantiations grows with the number of constraints and they are often more harmful, as they increase the search space, than of use. Z3 uses [relevancy filtering](#) to filter the Boolean assignments that are made visible to the theory solvers. It is inspired by search in semantic Tableau, where establishing satisfiability of a disjunction is split into separate branches that process the disjuncts in isolation.

In Z3, relevancy filtering is set up during clausification. The result of clausifying a Boolean formula is a set of *root* clauses and a set of *definition* clauses. The goal is to establish a labeling of literals as *relevant* such that

- the set of relevant literals satisfies *roots*
- there is a set of blocked literals that can be used to satisfy the clauses in *definitions* independent of their real assignment.

The idea is that the *definition* clauses are obtained from Tseitin transformation so they can be grouped by the blocked literal that was used to introduce them. For example, when clausifying $a \wedge b$ we have the clauses

$$\begin{aligned}
& \neg \boxed{(a \wedge b)} \vee a \\
& \neg \boxed{(a \wedge b)} \vee b \\
& \neg a \vee \neg b \vee \boxed{(a \wedge b)}
\end{aligned}$$

then the literal for $a \wedge b$ is blocked. And recursively for clauses introduced for a, b if they use Boolean connectives at top level.

Relevancy filtering applies a set of state transitions to update whether literals and terms are relevant and should be exposed to the theory solvers. The transitions for setting a literal ℓ and node n to relevant are:

- A literal ℓ is assigned and ℓ appears positively in a *root* clause C and no other literal in C are relevant.
- ℓ is inferred as a unit clause
- The equality $n \simeq n'$ is assigned and n' is relevant.

Roots and definitions are updated as well:

- When ℓ is assigned relevant, then all *definitions* where ℓ occurs negatively are added to *roots*.
- When a clause is added to *roots* then:
 - If it already contains a positive relevant literal it is skipped.
 - If there is a positive literal (but no other positive relevant literals), the positive literal is set relevant.
- When a clause is added to *definitions* and it contains a negative relevant literal, the clause is added instead to *roots*.
- When n is set relevant, arguments that don't represent Boolean connectives are set relevant. If-then-else terms are treated as Boolean connectives because they are compiled into clauses where, the clausal relevancy propagation takes effect.

4.8. The power of propagation

Efficient theory propagation can have a profound effect on overall efficiency of solving. Congruence closure is therefore tightly integrated with Boolean propagation: when equality reasoning can infer that an equality atom $n \simeq n'$ is true or false, it is useful to propagate this fact eagerly. Congruence closure in Z3 therefore has special handling of equality literals. An equality $n \simeq n'$ is both a term with the binary function $\simeq$ and an atom. Other atoms are also tracked. For example, if $p(t)$ is a unary predicate then the E-node data-structure also tracks the Boolean variable that is associated with $p(t)$. When t is equated to t' and $p(t')$ is assigned to true or false, then congruence closure propagates the assignment on to $p(t)$. The propagation is justified by the premises for the equality $t \simeq t'$. Any overhead of propagating within the E-graph is outweighed by the benefits it brings to pruning the search space.

4.9. Dynamic Ackermann reduction

Basic implementations of CDCL(T) as presented in Section 2.1 produce a proof system that has the power of regular resolution for propositional logic. The proof system is relatively weak even for EUF when search is limited over the space of initial set of literals from a formula.

Example 13.

The following clauses are unsatisfiable over the theory of equalities, but the shortest resolution proof that only uses the atomic formulas from the input is in the order of 2^{100} clauses. The resolution proof involves clauses with all combinations of b_i and c_i for i ranging from 0 to 99.

$$a_0 \not\simeq a_{100} \wedge \bigwedge_{0 \leq i < 100} (a_i \simeq b_i \vee a_i \simeq c_i) \wedge (a_i \simeq b_i \implies b_i \simeq a_{i+1}) \wedge (a_i \simeq c_i \implies c_i \simeq a_{i+1})$$

The proofs are linear if we admit clauses using fresh literals of the form

$$\begin{aligned}
(a_i \simeq b_i \wedge b_i \simeq a_{i+1} &\implies a_i \simeq a_{i+1}) \\
(a_i \simeq c_i \wedge c_i \simeq a_{i+1} &\implies a_i \simeq a_{i+1})
\end{aligned}$$

Z3 dynamically introduces such auxiliary clauses based on transitivity of equality and congruence rules of the form

$$t_1 \simeq s_1, \dots, t_k \simeq s_k \implies f(t_1, \dots, t_k) \simeq f(s_1, \dots, s_k)$$

The axioms are introduced for uninterpreted functions and related axioms are introduced for bit-vectors

$$\bigwedge_i (\text{bit2bool}_i(bv) \Leftrightarrow \text{bit2bool}_i(bv')) \implies bv \simeq bv'$$

where $(\text{bit2bool}_i(bv) \Leftrightarrow \text{bit2bool}_i(bv'))$ are compiled into new propositional literals for each bit position i of bit-vectors bv, bv' .

Dynamic Ackerman axioms can also, in theory, have the same effect for other theories, such as arithmetic.

Example 14.

The following arithmetic formula has no short vanilla CDCL(T) proof, but has a short proof if adding axioms for transitivity of inequality.

$$a_0 > a_{100} \wedge \bigwedge_{0 \leq i < 100} (a_i \leq b_i \vee a_i \leq c_i) \wedge (a_i \leq b_i \implies b_i \leq a_{i+1}) \wedge (a_i \leq c_i \implies c_i \leq a_{i+1})$$

In Z3, the EUF and bit-vector theories track the number of times transitivity of equality and congruences are applied in conflicts. It introduces the Ackermann axioms based on thresholds. A user can control the thresholds. An important question for applications is *when is dynamic Ackermann reduction useful?* The cost-benefit of introducing new literals into the search space has to be weighed. Importantly, for applications where the main expectation is that formulas are satisfiable, dynamic Ackermann reduction can be detrimental. Users can disable dynamic Ackermann reduction to gain significant performance improvement.

4.10. Iterative deepening for finite models

You can declare and reason about recursive functions with Z3. The SMTLIB2 standard introduces recursive function definitions and you can add recursive function declarations over the programmatic APIs.

In the following, we define two recursive functions `length` (the length of a list) and `nat-list` (a predicate whether a list of integers contains only non-negative integers).

Example 15.

```
(define-fun-rec length ((ls (List Int))) Int
  (ite ((_ is nil) ls) 0 (+ 1 (length (tail ls)))))

(define-fun-rec nat-list ((ls (List Int))) Bool
  (ite ((_ is nil) ls)
    true
    (and (>= (head ls) 0) (nat-list (tail ls)))))

(declare-const list1 (List Int))
(declare-const list2 (List Int))
(assert (> (length list1) (length list2)))
(assert (not (nat-list list2)))
(assert (nat-list list1))
```

To check satisfiability of a set of ground assertions of these functions, Z3 unfolds the definitions incrementally. It uses iterative deepening to control the unfolding depth. Internally, it uses *assumption literals* that are introduced by the theory solver for recursive functions to control the unfolding depth.³

The main CDCL(T) solver is now wrapped in an outer loop where assumption literals are supplied by theory solvers, and unsatisfiable cores that contain these assumption literals are presented to the theory solvers that can then extend their depth bounds. There is an advantage to maintaining finer-grained depth bounds, that is, separate assumption literals for each recursive function and each recursive branch within a function [40].

The theory of sequences also uses iterative deepening to control the search for finite length sequences. The sequence (string) theory also uses a `contains` predicate that captures when a sequence is a subsequence of another. Positive occurrences of `contains` can be reduced to solving string equalities, but negative occurrences of `contains` are more tricky. They are treated as recursive functions over sequences. Their unfolding is controlled by size bounds of the arguments to `contains`.

4.11. Incrementality

- External and internal scopes
- Undo-trail
- equi-satisfiability

5. Model Construction

The verdicts of satisfiability or unsatisfiability do not reveal by themselves any information about the *reason* for the outcome. Model object represent certificates for satisfiability, and proof objects or unsatisfiable subsets certificates for unsatisfiability. Furthermore, retrieving and using models is integral to many applications. For example, symbolic execution uses satisfying assignments to input variables of a path condition to produce new inputs to a program that explore a so-far uncovered execution branch. Propositional models are simply assignments of true/false to propositional variables. Similar, models of linear arithmetical formulas are assignments of integers or rationals to arithmetical variables. Model construction is more involved when considering a combination of theories, non-linear arithmetic formulas and quantified formulas.



5.1. Extracting models from satisfiable states

This section describes how models are constructed from a satisfiable state. A starting point for model construction in Z3 is a T-consistent assignment to atomic formulas such that all ground clauses are satisfied and the conjunction of assignments to atomic formulas are consistent with respect to all theories. Model construction then works by layers:

- For each E-node n it collects *dependencies* of E-nodes $n_1, \dots, n_k$ that must be assigned a value before n can be assigned a value. Dependencies are specific to theories. There are no dependencies when n ranges over Bool, Int, or Real.
 - When n is an array, the value of n depends on the values of all nodes $n'[n'']$ or $\delta(n')$ where $n \sim n'$. That is, the value of n as an array depends on the values of select and default terms where it occurs.
 - When n is an algebraic data-type headed by constructor $C(n_1, \dots, n_k)$, then the value of n depends on $n_1, \dots, n_k$.
 - When n is a finite sequence $[n_1, \dots, n_k]$, then the value of n depends on $n_1, \dots, n_k$.

Dependency collection is well-founded: for arrays, the dependencies come from the range of the array sort; for algebraic datatypes dependencies are sub-terms; for sequences dependencies are characters.

- Dependencies are then topologically sorted such that nodes without dependencies are processed first and assigned *values*.
 - Boolean nodes are evaluated by the SAT solver. They are assigned *true* or *false*. Nodes ranging over an uninterpreted sort are assigned to an abstract value that is distinct from values from other equivalence classes. Values of nodes with associated theory solvers are assigned by the solvers.
 - Int and Real nodes are assigned values assigned by the arithmetic solver. For non-linear arithmetic formulas, the values of a Real can be an algebraic numeral. Algebraic numerals are represented as a k 'th root to polynomials (whose coefficients can be algebraic numerals). They can be evaluated to decimal representations using an arbitrary degree of precision.
 - An array node n is assigned the value $\llbracket i \rrbracket \mapsto \llbracket n'[n''] \rrbracket \mid n'[n''] \in \text{Nodes}, n \simeq n' \rrbracket \cup [\text{else} \mapsto \delta(n') \mid n' \sim n]$.
 - The algebraic data-type $n \sim C(n_1, \dots, n_k)$ is assigned $C(\llbracket n_1 \rrbracket, \dots, \llbracket n_k \rrbracket)$.
- After all nodes have been assigned a value, model construction assigns interpretations to uninterpreted functions. Uninterpreted functions are assigned to lookup tables. f is assigned the value $\llbracket n \rrbracket \mapsto \llbracket f(n) \rrbracket \mid f(n) \in \text{Nodes} \rrbracket \cup [\text{else} \mapsto e]$, where e is one of the values from $\llbracket f(n) \rrbracket$. Z3 synthesizes interpretations of uninterpreted functions that go beyond finite lookup tables. In these cases, it mines quantified formulas for macros. For example, asserting $\forall x. x \geq 0 \implies f(x) \simeq x + 1$ creates the function interpretation $f(x) = \text{if } x < 0 \text{ then } f'(x) \text{ else } x + 1$ for f , where f' is a fresh function. The function f is replaced everywhere by the macro definition.

This completes the description of model extraction in Z3.

5.2. Characteristics of Models

We make the following observations

5.2.1. Models are explicit

Models provide explicit interpretations of terms and functions using values. We call such models *constructive symbolic models*. There are cases where first-order saturation theorem provers can determine that a set of first-order clauses are satisfiable, without presenting a symbolic model. Z3 does not include such saturation criteria. Model construction for the Horn clause solver SPACER also produces symbolic models.

5.2.2. Models are mainly based on values

Models produced by the solver map terms to values. Given a solution to $x \simeq y + 1$, the solver will assign x to a numeral that is one greater than the numeral assigned to y . Pre-processing, on the other hand, may choose to *solve* for x and record the solution $x \mapsto y + 1$. In this case, the model for x can be retrieved as $y + 1$. Z3 supports a setting to force $y + 1$ to be evaluated to a numeral if the user wants this.

add example

5.2.3. There are no deep knobs to control model values

Values to base types, such as Int, Real are dictated by the solvers. Values to terms of type Bool are controlled mainly by the SAT solver. Ensuring *diverse* model values for Booleans is partially possible by controlling options on the SAT solver [[@SoosUniSAT](#)]. There are no corresponding



knobs for values from theories. For instance, for linear arithmetic based on dual simplex, values are determined by an algorithm that seeks to find solutions along vertices in a polytope, which in layman's terms means finding values to bounded variables at their bounds. Thus, configurations such as `smt.arith.random_initial_value=true`, provide no strong assurances (furthermore, only the solver configured by setting `smt.arith.solver=2` for arithmetic responds to this configuration).

```
(set-option :smt.arith.solver 2)
(set-option :smt.random_seed 5)
(set-option :smt.arith.random_initial_value true)
(declare-const x Int)
(declare-const y Int)
(push)
(assert (> x y))
(check-sat)
(get-model)
```

A main knob for controlling model values are over optimization, νZ [NuZ]. By specifying one or more optimization objectives you can control model values of selected terms to be minimum or maximum.

add optimization example

5.2.4. Models include auxiliary definitions

Z3 includes auxiliary definitions in models. For example, if you define a recursive function, the recursive function definition is part of the model that is displayed in SMTLIB2 mode.

6. Certificates and Unsatisfiable Cores

6.1. Unsatisfiable Cores

6.2. Certificates

6.2.1. Legacy Proof Format

6.2.2. Proof Logs

The new core in Z3 produces *proof logs* that extend DRUP proofs. SAT solvers produce mainly DRUP proofs. The DRAT format (RAT = Reverse Asymmetric Tautology, while RUP = Reverse Unit Propagation, and D = with *Deletion*) is widely adopted for certifying proofs of SAT solvers. Z3 extends the DRUP format for SMT. It uses an extension of the SMTLIB2 format to record proof traces. We introduce it by an example using EUF.

From SMTLIB:

```
(declare-fun f (Int) Int)
(define-const $24 Int (f x))
(define-const $25 Int (f $24))
(define-const $26 Int (f $25))
(define-const $27 Bool (= $26 x))
(define-const $28 Bool (= $25 x))
(assume $27 $28)
(define-const $30 Int (f $26))
(define-const $31 Int (f $30))
(define-const $32 Int (f $31))
(define-const $33 Bool (= $32 x))
(assume (not $33))
(declare-fun rup () Proof)
(infer (not $33) rup)
(declare-fun euf (Bool Bool Proof Proof Proof Proof) Proof)
(declare-fun cc (Bool) Proof)
(define-const $42 Bool (= $32 $30))
(define-const $43 Proof (cc $42))
(define-const $40 Bool (= $31 $24))
(define-const $41 Proof (cc $40))
(define-const $38 Bool (= $30 $25))
(define-const $39 Proof (cc $38))
(define-const $36 Bool (= $24 $26))
(define-const $37 Proof (cc $36))
(define-const $34 Bool (not $33))
(define-const $44 Proof (euf $34 $28 $37 $39 $41 $43))
(infer (not $28) $33 $44)
(infer (not $28) rup)
(infer $27 rup)
(declare-fun euf (Bool Bool Proof Proof Proof) Proof)
(define-const $49 Bool (= $32 $26))
(define-const $50 Proof (cc $49))
(define-const $47 Bool (= $31 $25))
(define-const $48 Proof (cc $47))
(define-const $45 Bool (= $24 $30))
(define-const $46 Proof (cc $45))
(define-const $51 Proof (euf $34 $27 $46 $48 $50))
(infer $33 $51)
(infer rup)
```


It is possible to use Z3 to parse this format and pretty print it from Python:

```
assume(Or(f(f(f(x))) == x, f(f(x)) == x))
assume(Not(f(f(f(f(f(x)))))) == x))
infer(rup, Not(f(f(f(f(f(x)))))) == x))
infer(euf(Not(f(f(f(f(f(x)))))) == x),
      f(f(x)) == x,
      cc(f(x) == f(f(f(x)))),
      cc(f(f(f(x)))) == f(f(x))),
      cc(f(f(f(f(x)))) == f(x)),
      cc(f(f(f(f(f(x)))))) == f(f(f(x)))),
      Or(Not(f(f(x)) == x), f(f(f(f(f(x)))) == x))
infer(rup, Not(f(f(x)) == x))
infer(rup, f(f(f(x))) == x)
infer(euf(Not(f(f(f(f(f(x)))))) == x),
      f(f(f(x))) == x,
      cc(f(x) == f(f(f(f(x))))),
      cc(f(f(f(f(x)))) == f(f(x))),
      cc(f(f(f(f(f(x)))) == f(f(f(x))))),
      f(f(f(f(f(x)))) == x)
infer(rup, False)
```

The following script was used to pretty print the proof

```
from z3 import *

def on_clause(j, deps, c):
    if deps:
        print(deps[0], j, c, deps[1:])
    else:
        print(j, c)

s = Solver()
OnClause(s, on_clause)
s.from_file("name-of-file-with-proof-log.smt2")
```

There are the following layers to proof logs in Z3:

- **Propositional:** the main search engine is based on CDCL(T) where a SAT solver performs case analysis over a propositional abstraction of the SMT formula. The proof steps produced by the SAT solver are *rup* (reverse unit propagation) and *del* (clause deletion) steps.
- **Assumptions:** input assumptions are clauses that are based on the input formula. If Z3 uses pre-processing simplifications, then the assumptions are obtained from the original input by a sequence of pre-processing steps. Pre-processing can be turned off or justified separately using the legacy proof format. A tight integration of proofs from pre-processing and proof logs is currently not available.
- **Theory inferences:** Theory inferences are logged as inferences that are justified by theory axioms. Each theory supports its own set of theory axioms and the format for how theory axioms are justified is theory dependent. The solver for EUF combines inferences from theories using equality propagation.

EUF Solver core

The core of the CDCL(T) solver is a core based on EUF reasoning. It works as a dispatch between theory solvers and manages shared equalities.

The CDCL core integrates with the EUF solver when resolving conflicts using the following function.

$$explain : \mathcal{L} \rightarrow \mathcal{L}^*$$

It takes a literal that is assigned to true. The literal can also be $\perp$ (or false), in which case the state of the theory solvers is conflicting.

The requirement is that for a literal ℓ :

$$\bigwedge explain(\ell) \implies \ell \text{ is valid modulo background theories}$$

The *explain* function returns a list of literals, assigned by the CDCL core that justify the propagation of ℓ . The literal ℓ is associated with a reference to a theory justification or an EUF justification.

$$justification : \mathcal{L} \rightarrow \mathcal{J} \mid \text{EUF}$$

Each theory solver also implements a *explain* specific for the theories. The internal signature for *explain*:

$$theory-explain : \mathcal{J} \rightarrow \mathcal{L}^* \times \mathcal{E}^* \times ThJustification$$

where

$$\mathcal{E} ::= Expr \times Expr$$

The E-graph is used to explain equalities, or explain conflicts.

$$euf-explain : \mathcal{E} \cup \{\perp\} \rightarrow \mathcal{L}^* \times (\mathcal{J} \times \mathcal{E})^* \times Congruence^*$$



The list of constraints and equalities have their own justifications. The dependencies on equalities is well-founded so applying `explain` recursively produces just a list of literals.

We assume that each theory implements a *hint* function that logs a justification for the inference it made. It produces hint to justify the clauses

```
And(lits @ [a == b for (jst, (a, b)) in jsts]) => x == y   where (lits, jsts, cc) = euf-explain(
And(lits @ [a == b for (jst, (a, b)) in jsts]) => false   where (lits, jsts, cc) = euf-explain(
And(lits @ [a == b for (a, b) in eqs]) => L               where (lits, eqs, thjst) = theory-exp
```

The version of `explain` that is exposed to the SAT core is as follows:

```
explain(L) =
  match justification(L) with
  | EUF -> explain(false)
  | jst -> explain(jst)

explain(eq_or_false) =
  let (lits, jsts, cc) = euf-explain(eq_or_false)
  lits @ concat([explain(jst) for (eq, jst) in jsts])

explain(jst) =
  let (lits, eqs, thjst) = theory-explain(jst)
  lits @ concat([explain(eq) for eq in eqs])
```

To create certificates we need a decomposition.

```
proof(L) =
  match justification(L) with
  | EUF -> yield from proof(false)
  | jst -> yield from proof(jst)

proof(eq) =
  let (lits, jsts, cc) = euf-explain(eq)
  for (eq, jst) in jsts:
    yield from proof(jst)
  yield lits & [eq for (eq, jst) in jsts] => eq by cc

proof(jst) =
  let (lits, eqs, thjst) = theory-explain(jst)
  for eq in eqs:
    yield from proof(eq)
  yield lits @ eqs => literal(jst) by thjst
```

We introduced a new function

$$literal : \mathcal{J} \rightarrow \mathcal{L}$$

that produces either a literal propagation or conflict (where the literal is $\perp$).

Equalities that are propagated by theory propagation satisfy `euf-explain(eq) = ([], [(eq, jst)], [])`. For these cases there is no need to produce the implication `eq => eq`.

Implementation-wise, the justification for EUF is created on the fly when calling `euf-explain`. It is the product of combining literals, equations and the congruence closure steps. The justification for theory axioms is all encoded in `thjst`.

The two versions of `explain` can sometimes be combined: if the explanation for a literal propagation or a conflict only uses a single theory axiom or single EUF proof it is tempting to interleave the functionality. It gets dicier when it comes to controlling whether the E-graph should produce congruence closure justifications.

Proposition 1.

`Not(And(explain(L)))` can be proved using RUP from clauses produced by `proof(L)`.

We don't always need to justify a clause using RUP, but can use justifications directly.

Proposition 2.

Suppose that `justification(L) = EUF` and `euf-explain(false) = (lits, [], cc)`. Then the literals produced in the clause from `proof(L)` coincide with `explain(L)`. The same is the case when `justification(L) = jst` and `theory-explain(jst) = (lits, [], thjst)`.

6.2.3. EUF Axioms

There is only one kind of axiom for EUF inferences. The example indicated that EUF axioms are labeled as `euf` terms that take a single disequality, a set of equalities and a set of equalities under a function named `cc`. The idea is that the conjunction equalities implies the negated equality by taking the equivalence class of terms listed as equalities and under `cc`. Furthermore, each equality under a `cc` is justified from all previous equalities and `cc` terms before it by a single application of the congruence rule. Thus, if a `cc` equality is of the form `cc(f(x,y) == f(z,y))`, then `x, z` are equal modulo the previous seen equalities, and therefore `f(x,y)` is equal to `f(z,y)`. A variant of `cc`, called `comm`, is used for congruence of binary functions that are commutative. For example, `comm(f(x,y) == f(y,z))` is justified if `f` is commutative and the equality `x == z` (and `y == y`) was justified.

A validator for equality axioms can check that each equality and disequality under the `euf` justification corresponds to a literal in the clause that is inferred from the theory axiom. It can then check that the clause is a tautology for EUF by creating equivalence classes of the equalities and congruences it is supplied with. It checks that each application of a congruence is well-formed (the arguments to `f` are equivalent), and then it can check that the left and right side of the disequality are in the same equivalence class.

Example 16.

Consider the inference

```
euf(Not(f(f(f(f(f(x)))))) == x),
    f(f(f(x))) == x,
    cc(f(x) == f(f(f(x))))),
    cc(f(f(f(f(x)))) == f(f(x))),
    cc(f(f(f(f(f(x)))) == f(f(f(x))))))
```

The conjunction of `Not(f(f(f(f(f(x)))))) == x`, `f(f(f(x))) == x` is unsatisfiable. To justify this consider first the equality `f(f(f(x))) == x` and three applications of congruence closure to establish `f(f(f(f(f(x)))) == f(f(f(x))))`. By transitivity of equality, it follows that `f(f(f(f(f(x)))) == x` and therefore the negated equality is conflicting.

6.2.4. Arithmetic Axioms

Arithmetic solver produces (so far) theory axioms of the form

```
<rule> (<numeral> <literal>)*
```

where `<rule>` is one of `farkas`, `implied-eq`, `bound`, `cut`, `nla`.

A `<literal>` is an equality, inequality. For the `implied-eq` rule, the last literal is a disequality (negated equality).

The rules are assumed to be checkable using basic inferences. They are implemented within Z3 as part of a self-validator.

- `farkas`: The set of inequalities are multiplied by the numeral coefficients and added to form a single inequality. The equalities are solved by Gauss-Jordan elimination and the resulting solution is substituted into the inequality. The resulting inequality is expected to be infeasible.
- `bound`: all but the last inequality are treated similar to the Farkas rule. Then the resulting inequality is subjected to a cut rule and the last inequality is checked to be contradictory with the reduced inequality.
- `implied-eq`: the last literal is a negated equality. The equalities are solved using Gauss-Jordan elimination, and the inequalities are assumed to imply equalities. These implied equalities can be retrieved by applying exhaustive Fourier-Motzkin elimination. The resulting set of equalities obtained from the inequalities are then solved using Gauss-Jordan elimination. The solution is substituted into the last negated equality which should simplify to false.

There is currently no self-validator for `cut` and `nla` rules. There are a few sources for `cut` rules: Z3 applies a Gomory-Chvatal cut as described in the technical report associated with [24], and cuts based on [14] (see 3.3.2.5), as well as a GCD filter 3.3.2.1. We leave it to future work to establish self-validators for cuts. It is noteworthy that SMTInterpol's proof format does not directly have to deal with cuts. The SMTInterpol solver does not assert inequalities that come from cuts but instead creates a fresh atom. When the cut formula is created from a tableau where non-basic variables are at their bounds it can only have one truth assignment; the existing proof rules are sufficient to establish this. There are several sources for `nla` rules and an elaboration into certificates that are efficient to check is left to future work.

Example 17.

The inference

```
farkas(1, 0 == y + -1*z, 1, y + -1*u <= 0, 1, z + -1*u >= 1)
```

Justifies the clause

```
[Not(y + -1*u <= 0), Not(z + -1*u >= 1), Not(0 == y + -1*z)]
```

because normalizing the inequalities to $-y + u \geq 0$, $z - u \geq 1$ (multiplying both by 1) and adding them produces $z - y \geq 1$. However, applying Gauss-Jordan elimination to the equations produces a solution where z and y are equated and therefore the inequality $z - y \geq 1$ simplifies to $0 \geq 1$.

The inference

```
implied-eq(1, 0 == y + -1*u, 1, 0 == y + -1*u, 1, Not(y == u))
```

Justifies the clause

```
[Not(0 == y + -1*u), Not(0 == y + -1*u), y == u]
```

because Gauss-Jordan elimination applied to the set of equality premises produces a solution that equates y and u .

7. Quantifiers

The dedicated engines QSAT 2.4 and SPACER 2.3 handle classes of quantified formulas. The QSAT engine solves formulas using quantifier alternation over theories that allow quantifier elimination using model-based projection. The SPACER engine solves constrained Horn clauses over a limited set of theories (with arithmetic being central and including emerging support for algebraic datatypes, arrays and bit-vectors).

Quantifier reasoning is also available within the CDCL(T) core. Here it relies on *quantifier instantiation* engines. We describe the main engines in use.

The quantifier instantiation engines rely on auxiliary annotations with quantifiers. We elided them when first introducing terms in Section 4.1. The fields that are of relevance to quantifier instantiation are expanded below:

```
type Expr
  Var{ index : int; sort : Sort }
  App{ f : FuncDecl; args : list<Expr> }
  Quantifier{ b : Binder; decl : list<Declaration>; body : Expr;
    patterns : list<list<Expr>>;
    nopatterns : list<Expr>;
    weight : int;
    qid : symbol }
```

- A set of *multi* patterns are used by the E-matching engine to search for instantiations. Quantifiers can be manually annotated by patterns or if no patterns are annotated, z3 infers a set of patterns based on smallest subterms. More than one pattern might be needed to find a full instantiation of all quantified variables.
- A set of *non-patterns* that identify sub-terms that cannot be used as patterns. It provides a user with control over quantifier instantiation by the E-matching engine.
- A *weight* to provide prioritization of quantifier instantiation.
- A *quantifier identifier* (*qid*) which can be used to trace instantiations of a quantifier in logs.

7.1. E-matching [19]

Let us start with an example of quantifier instantiation using E-matching in action.

Example 18.

Consider the formula

$$\underbrace{(\forall x . f(g(x, c)) \simeq a)}_{p_\varphi} \wedge b \simeq c \wedge g(c, b) \simeq c \wedge f(b) \not\simeq a$$

The smallest subterm properly containing the bound variable x is $g(x, c)$ in $f(g(x, c)) \simeq a$. The only other occurrence of the function g is $g(c, b)$ in the equality $g(c, b) \simeq c$. Based on the ground equality $b \simeq c$ it follows $g(c, c) \sim g(c, b)$ so we can instantiate x by c and infer

$$p_\varphi \rightarrow f(g(b, c)) \simeq a$$

The formulas are now ground unsatisfiable.

Quantifier instantiation using E-matching in Z3 serves a sweet spot by producing instantiations that take congruences of equalities into account, quickly, across a large set of terms and incrementally.

7.1.1. E-matching - basic algorithm

E-matching is a first-order matching algorithm that takes a congruence closure into account when checking for equality. We can describe a basic E-matching algorithm as taking

- a *pattern* that may contain variables,
- an E-node n , with set of equal nodes accessed as *n.siblings*,

and producing substitutions that represent the set of matches for n with respect to *pattern*.

```
def match(pattern, n, θ):
  pattern = patternθ
  if pattern == n:
    yield θ
  elif is_var(pattern):
    yield θ[pattern ↦ n]
  else:
    f(patterns) = pattern
    for f(terms) in n.siblings:
      # e.g., with same head function symbol f
      for θ' in matches(patterns, terms, θ):
        yield θ'
```

```

def matches(patterns, terms,  $\theta$ ):
    if not patterns:
        yield  $\theta$ 
        return
    for  $\theta'$  in match(patterns[0], terms[0],  $\theta$ ):
        for  $\theta''$  in matches(patterns[1:], terms[1:],  $\theta'$ ):
            yield  $\theta''$ 

```

The same algorithm can be presented in equational form with an extra argument S that accumulates substitutions.

$$\begin{aligned}
 \text{match}(x, t, S) &= \{ \theta[x \mapsto t] \mid \theta \in S, x \notin \theta \} \\
 &\quad \cup \{ \theta \mid \theta \in S, x \in \theta, \theta(x) \in t.\text{siblings} \} \\
 \text{match}(c, t, S) &= \emptyset \quad \text{if } c \notin t.\text{siblings} \\
 \text{match}(c, t, S) &= S \quad \text{if } c \in t.\text{siblings} \\
 \text{match}(f(ps), t, S) &= \bigcup_{f(ts) \in \text{cc}(t)} \text{match}(ps_n, ts_n, \dots, \\
 &\quad \text{match}(ps_1, ts_n, S))
 \end{aligned}$$

7.1.2. E-matching - code trees

The straight-forward E-matching algorithm is expensive when it is invoked from scratch for every possible candidate of pattern and node during search. To share work across patterns and terms, Z3 compiles patterns into a *code tree* data-structure that contains a dynamically compiled set of instructions to perform matching. The instructions can be regarded as a partial evaluation of the naive algorithm for fixed patterns. The instruction set allows merging code sequences from different patterns by sharing common prefixes. Substitutions are stored in registers, backtracking just updates the registers.

The abstract matching machine uses an array of registers $\text{reg}[i]$ to store sub-terms. Instructions are stored at program counters and each instruction refers to a next program counter:

Consider the following pattern

$$f(X, g(X, a), h(Y), b)$$

It compiles into a sequence of instructions as follows:

PC	Instruction	
pc_0	init(f, pc_1)	add arguments of f to registers 1-4
pc_1	check($4, b, pc_2$)	check if $\text{reg}[4]$ is congruent to b
pc_2	bind($2, g, 5, pc_3$)	bind terms in $\text{reg}[2]$ with g to 5 – 6
pc_3	compare($1, 5, pc_4$)	check if $\text{reg}[1]$ equals $\text{reg}[5]$
pc_4	check($6, a, pc_5$)	check if $\text{reg}[6]$ is congruent to a
pc_5	bind($3, h, 7, pc_6$)	bind terms in $\text{reg}[3]$ with h to 7
pc_6	yield($1, 7$)	output binding from $\text{reg}[1], \text{reg}[7]$

Let us trace the instructions for the term $f(h(a), g(h(c), a), h(c), b)$

PC	Instruction	$f(h(a), g(h(c), a), h(c), b)$
pc_0	init(f, pc_1)	$\text{reg}[1 : 4] \leftarrow h(a), g(h(c), a), h(c), b$
pc_1	check($4, b, pc_2$)	$\text{reg}[4] = b$
pc_2	bind($2, g, 5, pc_3$)	$\text{reg}[5 : 6] \leftarrow h(c), a$
pc_3	compare($1, 5, pc_4$)	$\text{reg}[1] = h(a) \neq h(c) = \text{reg}[5]$
pc_4	check($6, a, pc_5$)	
pc_5	bind($3, h, 7, pc_6$)	
pc_6	yield($1, 7$)	

If we instead use the term $f(h(a), g(h(a), a), h(c), b)$, we get the following execution

PC	Instruction	$f(h(a), g(h(a), a), h(c), b)$
pc_0	init(f, pc_1)	$\text{reg}[1 : 4] \leftarrow h(a), g(h(a), a), h(c), b$
pc_1	check($4, b, pc_2$)	$\text{reg}[4] = b$
pc_2	bind($2, g, 5, pc_3$)	$\text{reg}[5 : 6] \leftarrow h(a), a$
pc_3	compare($1, 5, pc_4$)	$\text{reg}[1] = h(a) = h(a) = \text{reg}[5]$
pc_4	check($6, a, pc_5$)	$\text{reg}[6] = a$
pc_5	bind($3, h, 7, pc_6$)	$\text{reg}[7] \leftarrow c$
pc_6	yield($1, 7$)	$X \leftarrow h(a) = \text{reg}[1], Y \leftarrow c = \text{reg}[7]$

Patterns that share common (top-down) term structure can share code sequences.

- This saves on common work.
- Use the choice instruction to branch when patterns end up having different sub-terms.

Other instructions are possible,

- forward pruning: lookahead multiple function symbols in immediate subterms before diving into any subterm.

- to deal with *multi-patterns*, when matching more than one pattern at a time.

7.1.3. Inverted path indexing

During search, congruence classes are merged and we would like to learn which merges should trigger new pattern match attempts. Z3 uses a filter called an *inverted path index* that tracks when a pattern is potentially useful for matching.

Example 19.

The pattern $f(X, g(X, a), h(Y), b)$ contains the subterm $g(X, a)$ in position 2. The merge between n_1 and n_2 can potentially trigger a new match when $n_1.siblings$ contains a node labeled by g , and n_2 has a parent labeled by f , and occurs as a second position of f .

Z3 pre-computes an index based on f, g pairs like these and check if it is triggered for some f, g pair and pattern when nodes n_1, n_2 are merged.

7.1.4. Relevance Propagation

Z3 uses *relevance propagation* to harness quantifier instantiation. Relevance propagation attempts to emulate semantic tableau rules on top of the CDCL(T) engine. In a semantic tableau, a formula is decomposed based on top-level connectives. Depending on the connectives, it creates one or more branches. Different branches then contain only atomic subformulas that are relevant to establishing that the formula in the root is true. We list tableau rules for a minimal set of logical connectives as inference rules:

$$\begin{array}{c}
 \frac{\bigvee_{i=1}^k \phi_i}{\phi_1 \mid \dots \mid \phi_k} \vee \\
 \frac{\phi \leftrightarrow \psi}{\phi, \psi \mid \neg\phi, \neg\psi} \leftrightarrow \\
 \frac{\text{ite}(\phi, \psi, \theta)}{\phi, \psi \mid \neg\phi, \theta} \text{ite}
 \end{array}
 \quad
 \begin{array}{c}
 \frac{\neg \bigvee_{i=1}^k \phi_i}{\neg\phi_1, \dots, \neg\phi_k} \neg\vee \\
 \frac{\neg\neg\phi}{\phi} \neg\neg \\
 \frac{\neg(\phi \leftrightarrow \psi)}{\phi, \neg\psi \mid \neg\phi, \psi} \leftrightarrow \\
 \frac{\neg\text{ite}(\phi, \psi, \theta)}{\phi, \theta \mid \neg\phi, \psi} \neg\text{ite}
 \end{array}$$

The rules are read top-down: To establish satisfiability of the formula at the root tableau rules create branches, separated by $\mid$. A branch is closed if it contains a pair of complementary subformulas (one is the negation of the other). Otherwise a branch is open.

Relevancy propagation follows the flow of the inference rules:

- Let ϕ be shorthand for $\bigvee_i \phi_i$. If ϕ is assigned true and is marked as relevant, then the first child ϕ_i that gets assigned true is marked relevant. If ϕ is assigned false and is marked as relevant, then all children are marked relevant.
- Let ψ be shorthand for $(\phi_1 \leftrightarrow \phi_2)$. If ψ is marked as relevant, then both ϕ_1 and ϕ_2 are marked as relevant.
- Let θ be $\text{ite}(\theta_1, \theta_2, \theta_3)$. If θ is marked as relevant, then if θ_1 is assigned to true (false), then θ_2 (θ_3) is marked as relevant.

7.1.5. The life-cycle of E-matching.

When formulas are parsed and asserted, they are converted into clauses. Quantified formulas are treated as atomic formulas. They are opaque relative to the CDCL(T) engine and treated as if they are propositional atoms. It is only when quantified formulas are asserted, either at base level during search or during backtracking, that E-matching kicks in.

The E-matching engine reacts to the following updates to the state during search:

- A universally quantified formula $\forall x, \pi(x) . \varphi$ with pattern π is assigned to true. In this case, the pattern $\pi(x)$ is compiled into a sequence of matching instructions and the instructions are inserted into the current code-tree. Upon backtracking, when the formula $\forall x, \pi(x) . \varphi$ is no longer assigned to true, the instructions from the pattern π are removed.
- When a term $f(t)$ becomes relevant, the matching engine triggers a lookup on the function f to determine if $f(t)$ can potentially trigger a new match.
- When two equivalence classes (of relevant terms) are merged by congruence closure, the E-matching engine examines whether the equivalence class merge can trigger a match.

Note that all state updates are reversed during backtracking. Only lemmas that were created as a side-effect of conflicts survive backtracking and become the seeds for what quantifiers, terms and equivalences that survive.

When E-matching triggers a quantifier instantiation it creates a formula $((\forall x . \varphi[x]) \implies \varphi[t])$, which is universally valid. The formula is asserted as an axiom. The formula is never garbage collected as long as search stays within the backtracking scope of where the instance was created. It gets garbage collected during backtracking if the instantiation happens within a nested search level.

7.1.6. Prioritizing Quantifier Instantiation

The *weight* field on quantifiers is used to assign a priority to quantifier instantiation. Quantifiers with *weight* set to 0 are instantiated without delay. Quantifiers with higher weights are inserted into a quantifier instantiation queue. The queue is emptied based on a set of rules:

- The *cost* of a quantifier is computed as a function of the *weight* other attributes using an expression that can be configured by setting the parameter `smt.qi.cost`. The default value of `smt.qi.cost` is $(+ \text{ weight } \text{ generation})$, where *weight* is obtained from the quantifier and *generation* keeps track of instantiation depths. All terms have initially generation 0, terms created as a by-product of quantifier instantiation have 1 plus the maximal generation of previous terms. Other parameters that can be used to encode quantifier priorities include
 - `instances` number of instances of current branch
 - `size` number of sub-expression of the quantifier
 - `depth` depth of quantified expression
 - `vars` number of bound variables
 - `pattern_width` number of patterns in multi-pattern annotation
 - `total_instances` the number of existing instances based on the quantifier
 - `scope` the scope level of search (how many case splits)
 - `cs_factor` a case split factor computed by considering number of disjuncts in a quantified body
- Quantifiers are instantiated eagerly if their cost is below a configurable `smt.qi.eager_threshold`.
- A partial checker can be enabled by setting `smt.qi.quick_checker` to 1 or 2 to promote instantiation where quantified instantiations are known to close a branch.
- Quantifiers whose cost are above the eager threshold and does not pass a quick checker are inserted into a *lazy* queue.

The *lazy* queue is instantiated on *final* checks. It uses a cost threshold to iteratively increase the number of instances that are processed.

- All quantifiers with cost below `smt.qi.lazy_threshold` are instantiated during final check.

7.2. Model-based Quantifier Instantiation [10, 20, 26, 45]

Model-based Quantifier Instantiation, MBQI, is based on a simple concept of using ground models to model check quantifiers, and extracting substitutions from quantifiers that fail the model check. Conceptually, MBQI uses the procedure:

Check $\psi \wedge \forall x. \varphi[x]$

```

while  $\psi$  is SAT with model  $M$ :
  if  $\neg\varphi^M[x]$  is UNSAT return SAT
   $M \leftarrow$  model for  $\neg\varphi^M[x]$ 
  find  $t$ , such that  $x \notin t, t^M = x^M$ .
   $\psi \leftarrow \psi \wedge \varphi[t]$ 
return UNSAT

```

where t^M is t partially evaluated using interpretation M .

Example 20.

Assume we have a model $M := [y \mapsto 3, f(x) \mapsto \text{if } x = 1 \text{ then } 3 \text{ else } 5]$ and term $t := y + f(y) + f(z)$. Then the specialization of t with respect to M is

$$t^M = 8 + \text{if } z = 1 \text{ then } 3 \text{ else } 5$$

MBQI offers a powerful setting to perform quantifier reasoning when the search space for instantiation terms is finite. This is the case for several fragments: EPR, UFBV, Array property fragment, Essentially UF. MBQI can also be combined with a set of templates to search over instantiation terms.

7.2.1. EPR

Effectively Propositional Reasoning, also known as Bernays-Schoenfinkel-Ramsey class comprises for formulas with relations, equalities, universally quantified variables, constants but no functions. Formally, it can be described as a set of formulas of the form:

$$\begin{aligned}
 \text{EPR} &::= \exists e_1 \dots e_n \forall u_1 \dots u_m F \\
 F &::= \bigwedge_i C_i \\
 C &::= \bigvee_j L_j \\
 L &::= A \mid \neg A \\
 A &::= p(\vec{V}) \mid V = V' \\
 V &::= e_i \mid u_j
 \end{aligned}$$

EPR is decidable, as seen from a brute-force decision procedure:

- Skolemize
 - $\forall u_1 \dots u_m F$, the $e_1, \dots, e_n$ are free constants
- Instantiate
 - $\bigwedge_{\theta} F\theta$
 - where θ ranges over all bindings of u_i to e_j .
- Check ground SAT
- Ground SAT implies finite model of size at most n .

MBQI can be used to harness the set of groundings.

- Skolemize
 - $\forall u_1 \dots u_m F$, the $e_1, \dots, e_n$ are free constants
- Models for $\neg F$ bind variables $u_1, \dots, u_m$ to free constants
- The number of possible such models is bounded by m^n .

7.2.2. UFBV [45]

$F ::= \exists e_i : bv[n] . F \mid \forall u_j : bv[n] . F \mid F \wedge F \mid C$
 $C ::= \bigvee_j L_i$
 $L ::= A \mid \neg A$
 $A ::= p(\vec{T}) \mid T = T' \mid \text{bvl}_e(T, T') \mid \dots$
 $T ::= f(\vec{T}) \mid e_i \mid u_j \mid \text{bit-vector expression}$
 $bv[n] ::= \text{bit-vector of length } n$

UFBV is decidable:

- All variables range over finite domains.
- Quantifier-free fragment is not only NP hard, it is NEXPTIME hard.
 - QF-UFBV can be encoded into EPR. [42]
- Quantified fragment is another complexity jump.
- BV - quantifier elimination [31–33]
- UFBV using MBQI [45]
 - Use templates to find general instances

7.2.3. Map Property Fragment [11]

$MP ::= \exists \vec{e} . \forall \vec{u} . \bigwedge (G \Rightarrow F)$
 $G ::= G \wedge G \mid A_G$
 $A_G ::= T[u] \simeq T[u] \mid T[] \not\simeq T[u]$
 $T[x] ::= x \mid e$
 $F ::= F \vee F \mid A_F \mid \neg A_F$
 $A_F ::= T[a[u]] \simeq T[a[u]]$

where a are arrays and $T[]$ denotes just e .

Example 21.

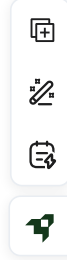
Let us express the following in the Map Property Fragment:

- a is the constant e -array.
- a is equal to b except at index v .
- c can only have two values, e_1 or e_2 .

$\forall u . a[u] \simeq e$
 $\forall u . u \neq v \Rightarrow a[u] \simeq b[u]$
 $\forall u . c[u] \simeq e_1 \vee c[u] \simeq e_2$

7.2.4. Array Property Fragment [11]

The Array Property Fragment extends the Map Property Fragment by allowing limited arithmetic.



$$\begin{aligned}
AP &::= \exists \vec{e}. \forall \vec{u}. \bigwedge (G \Rightarrow F) \\
G &::= G \wedge G \mid A_G \\
A_G &::= T_G \geq T_G \\
T_G &::= u \mid T[] \\
T[x] &::= x \mid e \mid k \times T[x] \mid T[x] + T[x] \\
F &::= F \vee F \mid A_F \mid \neg A_F \\
A_F &::= T_F \geq T_F \mid R(T_F, \dots, T_F) \\
T_F &::= T[a[u]] \\
k &::= \text{a numeric constant} \\
R &::= \text{a predicate}
\end{aligned}$$

The claim is that the array property fragment admits a finite set of instantiations.

- Given formula $\varphi := \forall \vec{u}. \bigwedge (G \Rightarrow F)$ with $\vec{e}$ free.
- Let $\mathcal{I} = \{c_1, \dots, c_m\}$ be set of a set of $T[] \cup \{0\}$ from φ .

Definition 1.

The set $\mathcal{I}$ is a *sufficient set of instances* for φ , if

$$\forall u. \bigvee_{c \in \mathcal{I}} \bigwedge_{A_G} (A_G \Rightarrow A_G[c/u])$$

- In other words, for every possible instantiation of u there is a c that satisfies at least the same combination of guards.

Proposition 3.

If φ admits a finite sufficient set of instances $\mathcal{I}$, it can be evaluated using those.

Proposition 4.

Formulas in the array and map property fragments admit a sufficient set of instantiations.

Proposition 5.

MBQI is a decision procedure for the array and map property fragments by using the instantiation sets computed from the formulas.

Instantiations based on $\mathcal{I}$ are sufficient for establishing unsatisfiability. To establish satisfiability, MBQI updates models that are based on how uninterpreted functions evaluate on arguments in $\mathcal{I}$ to how they evaluate on all integers. Models are updated in ways that do not change the interpretation of quantifier-free formulas. This is accomplished by keeping interpretation of ground terms the same, but adjusting the interpretations of functions for values that are not in the ground interpretation. For arithmetic and bit-vectors it updates functions to be piecewise constant between values from $\mathcal{I}$. In other words, suppose $\mathcal{I} = \{c_1, c_2, \dots, c_k\}$, and the set happens to be sorted such that $\mathcal{M}(c_1) \leq \mathcal{M}(c_2) \leq \dots \leq \mathcal{M}(c_k)$, and suppose f is a function from integers to integers. The ground model $\mathcal{M}$ interprets $f(c_1), \dots, f(c_k)$ and has some dummy interpretation for f for values outside of $\{\mathcal{M}(c_1), \mathcal{M}(c_2), \dots, \mathcal{M}(c_k)\}$. Model fixing changes the dummy interpretation of f to a piecewise constant interpretation by using the values from this set. For $x < \mathcal{M}(c_1)$ it sets the interpretation of $f(x) = \mathcal{M}(f(c_1))$. For x between $\mathcal{M}(c_i)$ and $\mathcal{M}(c_{i+1})$ it has the option to set the the interpretation of $f(x)$ to either $\mathcal{M}(f(c_i))$ or $\mathcal{M}(f(c_{i+1}))$.

Z3 represents the updated interpretations to the function f using *projection functions*. In other words, it uses the original interpretation of f as an interpretation to a fresh function f_{aux} , and then interprets $f(x)$ as $f_{aux}(f_\pi(x))$, where f_π is a projection function. The projection function maps an integer x to the bound c_i or c_{i+1} , where $\mathcal{M}(c_i) \leq x \leq \mathcal{M}(c_{i+1})$.

Models are also updated by detecting macro patterns in quantified formulas. For example, a quantified formula $\forall x. f(x) = t$, where f is not used in t defines f as a macro. The interpretation of $f(x)$ is a function of t . Z3 uses several techniques to extract macro interpretations from quantified formulas. After exhausting the search for macro interpretations it fixes the models for remaining functions to be piecewise constant.

7.2.5. Essentially Uninterpreted Fragment [26]

The Array Property Fragment can be seen as an instance of a class of formulas, where instantiations can be extracted based on a set of grammar rules that are extracted from the formulas.

This fragment thus applies to a wider range of formulas than the syntactic array property fragment. It also captures fragments studied in different contexts:

- the list property fragment by McPeak and Nacula
- several locally finite theories by Stokkermans et. al.

Example 22.

```

(set-option :smt.mbqi true)
;; f an g are "streams"
(declare-fun f (Int) Int)
(declare-fun g (Int) Int)

```



```
;; the segment [a, n + a] of stream f is equal to the segment [0, n] of stream g.
(declare-const n Int)
(declare-const a Int)
(assert (forall ((x Int)) (= (and (<= 0 x) (<= x n))
                             (= (f (+ x a)) (g x)))))

;; adding some constraints to a
(assert (> a 10))
(assert (>= (f a) 2))
(assert (<= (g 3) (- 10)))

(check-sat)
(get-model)
```

7.3. Quantifier Elimination through Macro Introduction

An important ingredient in quantifier reasoning is found in pre-processing. Pre-processing techniques may identify quantified assertions as macro definitions. Z3 contains several heuristics for identifying macros.

7.3.1. Tactic macro-finder

The tactic finds implicit macro definitions in quantifiers. A main instance of a macro is an equality that defines a function f using some term t that does not contain f . Other instances of macros are also recognized by the macro finder.

- $(\text{forall } (x) (= (f x) t))$
- $\text{not } (= (p x) t)$ is recognized as $(p x) = (\text{not } t)$
- $(\text{iff } (= (f x) t) \text{ cond})$ rewrites to $(f x) = (\text{if cond } t \text{ else } (k x))$
 - add clause $(\text{not } (= (k x) t))$
- $(= (+ (f x) s) t)$ becomes $(= (f x) (- t s))$
- $(= (+ (* -1 (f x)) x) t)$ becomes $(= (f x) (- (- t s)))$

7.3.2. Tactic elim-predicates

The elim-predicates subsumes macro-finder (and the quasi-macro tactic that we skip describing). In addition to the macro-finder macros it identifies rewrite opportunities of the form

- $(\text{assert } (\text{forall } ((x \text{ Int}) (y \text{ Int})) (= (f x y (+ x y)) (* 2 x y))))$ that implies that f can be treated as a function $(f x y z) = (\text{if } (= z (+ x y)) (* 2 x y) (f' x y z))$
- Predicates that occur at most in every clause and such that the cross-product of positive and negative occurrences is small relative to the original set of occurrences are eliminated. The elimination process was introduced by Davis and Putnam in a seminal paper that preceded the Davis Logeman Loveland backtracking SAT procedure. We illustrate the functionality of elim-predicates with an example

```
(declare-fun f (Int Int Int) Int)
(declare-fun p (Int) Bool)
(declare-const a Int)
(declare-const b Int)

(assert (forall ((x Int) (y Int)) (= (f x y (+ x y)) (* 2 x y))))
(assert (p (f 8 a (+ a 8))))
(assert (not (p (f 0 a (+ a 8)))))
(assert (not (p (f 2 a (+ a 8)))))
(assert (not (p (f 1 a (+ a b)))))
(apply elim-predicates)
```

The result of the tactic is a goal of the form:

```
(goal
  (not (= (* 16 a) (f!0 0 a (+ 8 a))))
  (not (= (* 16 a) (f!0 2 a (+ 8 a))))
  (let ((a!1 (= (* 16 a) (ite (= 1 b) (* 2 a) (f!0 1 a (+ a b))))))
    (not a!1))
  :precision precise :depth 1)
```

8. Tactics

Z3 exposes more than 100 *tactics* for solving or decomposing goals into one or more sub-goals. Tactics can be composed to form new more powerful tactics that meet needs of custom solving tasks. Tactics, in the context of Z3, were introduced in [18], there is a set of samples for using tactics [online](#), and many tactics are [summarized with examples](#). While we have to refer to the online sources for what various tactics accomplish and how to configure them we will here summarize an ontology of tactics and provide a high-level guide towards using main tactics.



The first aim of a user is likely well captured by an aim to gain *insights into selection of suitable tactics for specific problems*. We will not here attempt to catalog a mapping from problem classes to tactic combinations, the space is too open ended and a partial categorization requires significant effort. The reality is that selecting suitable tactics remains an experimental craft: for a given class of problems a user can try available tactics on small instances to measure their effects to deduce overall effects. Tactic selection has even been the focus of integration with machine learning tools [1]. In this guide we will try to give the reader tools to understand tactics in context to guide experimentation.

8.1. A use case of tactics from Alive2

Alive2 [36] is a translation validation tool for the LLVM compiler. It works by snapshotting the code before and after optimizations and proving that the optimized code refines the original code. Alive2 uses Z3 to solve all the logic formulas it generates.

Alive2 uses a combination of features and logics of Z3: bit-vectors, arrays, lambdas, floating-point numbers, UFs, and quantifiers. Sometimes it uses all of these in a same query. This is an unusual mix of theories, that is not observed in the SMT-LIB benchmarks. Since Z3's tactics are mostly tuned for the SMT-LIB benchmarks, it is natural that Z3's standard tactic pipelines are not suitable for Alive2.

Using tactics in Alive2 was really necessary! For some of the larger benchmarks, using its custom tactic is an order of magnitude faster than using Z3's standard tactic pipelines. Another thing to keep in mind is that not all of Z3's tactic are hardened to be used from the API. For example, some tactics may not respect resource limits (e.g., time, memory) or have a super-linear cost for certain formulas. Hence, using a small set of hardened tactics is advisable for production applications.

For example, Alive2 queries have a lot of UFs. However, many of those UFs have constant arguments or they become constant after using the `simplify` tactic. It is then very useful to run the `reduce-args` tactic to remove arguments of UFs that are constant, or even to transform UF applications into variables (e.g., `f(#x42)` is replaced with `f!1`).

After removing UFs, other tactics that don't work with UFs can be used now. For example, Alive2 runs `elim-uncnstr` to remove single uses of variables, either boolean, or BV variables used in a single constraint.

Alive2 also produces a lot of quantifiers. Many of these formulas are not too complicated and can be solved through Z3's simple quantifier elimination tactic (`qe-light`). Z3 also has a more advanced quantifier elimination tactic (`qe`), but it is too slow for our large formulas and often doesn't pay off.

In the compiler world, the phase ordering problem is well known: finding the best order of optimizations for each program is a very hard problem. The problem space is huge given that the number of optimizations is large, and it's often beneficial to run some optimizations more than once. The same issue arises with SMT solvers: the right sequence of tactics depends on the problem instance. Similarly, some tactics should be run multiple times.

For example, Alive2 runs `qe-light` twice. The first run exposes several opportunities for other tactics (like removing unconstrained expressions). Running these tactics, then allows `qe-light` to eliminate a few more quantifiers.

Most of Z3's tactics are not useful for Alive2, which emphasizes the need to customize the tactic pipeline. First, some tactics target theories that Alive2 doesn't use. Second, some tactics are only useful when formulas are written in a way that Alive2 will never generate. For example, the `demodulator` tactic essentially instantiates axioms (a particular kind of forall quantified formulas) present in the formula. Alive2's `vcgen` algorithm already instantiates all the relevant axioms, as it has more domain information, so it can do a better job. Therefore, Alive2 doesn't benefit from Z3's `demodulator` tactic.

Finally, some of Z3's tactics are expansive: they increase the size of the formula, hoping the rewritten formula will simplify further later on. For example, the `blast-term-ite` tactic will rewrite `a + ite(c, b, d)` into `ite(c, a + b, a + d)`. This can open opportunities for further simplification, but it can also increase the size of the formula exponentially. Alive2, in particular, often generates formulas with deeply nested `ite` expressions, hence it's not possible to use such a tactic. Eager bit-blasting, as mentioned before, doesn't yield good results as variables tend to have large bit-widths.

Besides tactics, there are also the SMT solver (and tactics) parameters. For example, Z3 has two algorithms for quantifier instantiation: E-matching and MBQI. These algorithms have different characteristics and each will perform best with certain formulas and queries. It's left to the user to pick the best algorithm (through the `smt.ematching` config option).

Another thing to consider is theories' options. Some theories have options to do small changes to their semantics to improve performance. For example, when converting a float number to a bit-vector, Z3 internally uses a UF to represent an arbitrary NaN payload. However, if this behavior is irrelevant to the application (e.g., because NaNs are irrelevant or because NaNs are



already handled properly]), it can be disabled by setting `rewriter.hi_fp_unspecified` to `true`. Other cases include, for example, bit-vector division by zero.

To summarize, Alive2's main tactic and solver options are as follows:

```
(set-option :model.partial true)
(set-option :smt.ematching false)
(set-option :smt.mbqi.max_iterations 1000000)
(set-option :memory_high_watermark 2147483648) ; soft 2 GB memory limit
(set-option :rewriter.hi_fp_unspecified true)
(and-then simplify propagate-values simplify elim-uncnstr qe-light
           simplify elim-uncnstr reduce-args qe-light simplify smt)
```

8.2. A classification of tactics

All tactics create at least one sub-goal. If a tactic is able to determine satisfiability of a goal, the subgoal contains no assertions. If it determines unsatisfiability, the subgoal contains the assertion *false*. Otherwise, the subgoal is a set of assertions. Several subgoals can be viewed as a subgoal containing the disjunction of assertions from each subgoal. The relationship between the original goal and subgoals is determined by what the tactic accomplishes. The tactic may produce a subgoal that either

relationship	description	example
preserves equivalence	the subgoal is equivalent to the original goal	simplify
preserves satisfiability	the subgoal is equi-satisfiable, but not equivalent	solve-eqs
weakening	if the subgoal is unsatisfiable, the original goal is too	sat
strengthening	if the subgoal is satisfiable, the original goal is too	nla2bv

The `simplify` tactic performs rewriting simplification steps. The rewrites implemented within `simplify` are equivalence preserving. The `solve-eqs` tactic eliminates uninterpreted constants when they can be written as solutions to equations. For example $x + 1 = 2y$ allows solving x as $2y - 1$. It removes x from the subgoal which is therefore not equivalent to the original goal. The `sat` tactic solves satisfiability of a propositional abstraction of the original goal. The `nla2bv` tactic approximates reals and integers by bit-vectors and therefore constrains the values of possible solutions to a small space that can be expressed using the bit-vectors.

8.3. Model Converters

Tactics that preserve satisfiability, but not equivalence, produce as a side-effect a *model converter*. The model converter is maintained internally. It can be accessed on the subgoal to convert a model of the subgoal to a model of an original goal. Within the satisfiability preserving tactics there is a further distinction between the model converters associated with tactics. We classify them next.

effect on models	example input	example output	model converter
rigid constrained	$x = t$	<i>true</i>	$x = t$
under-constrained	$(x = t \wedge \varphi) \vee \psi$ $x \notin \psi$	$\varphi[t/x] \vee \psi$	$x = t$
under-constrained	$x \leq y, x \leq z, y \leq u, \varphi$ $x, y \notin \varphi$	φ	$u = y, z = x, y = x$
over-constrained	$p(x), p(y), p(z)$	$x \leq y \leq z$ $p(x), p(y), p(z)$	n/a

The base functionality of `solve-eqs` tactic is an example of a rigid constrained tactic. It includes the substitution $x = t$ in the model converter. Rigid constrained tactics have the property that the model converter can be applied as a substitution to new formulas while preserving satisfiability. In other words, rigid constrained tactics can be applied incrementally without undoing the effect of a transformation. The `tseitin` tactic is also rigid constrained. It introduces fresh variables to encode Boolean connectives, but the evaluation of those variables are hidden from the model of the original formula. Thus, there is another category of model conversions (removing interpretations of variables from the model for the original formula) that can be applied incrementally. The `solve-eqs` tactic has an option, which is enabled by default, to solve equations within disjunctive context. If x is a variable that occurs within a single disjunction and nowhere else, it can be solved for as well. The resulting formula is weaker than the original formula because it allows solutions to x and $\varphi[t/x]$ where $x \neq t$. To apply under-constrained tactics incrementally it is necessary to replay formulas that were modified by the transformations. Another example of an under-constrained tactic is `elim-uncnstr`. It solves for variables that occur uniquely in a goal. For example, if the only occurrence of u is in the inequality $y \leq u$ we can solve for u by setting $u = y$. At this point, both y and z are unique and we can solve for y, z by setting them to x . The `elim-uncnstr` tactic is more general than our simplified example suggests, it can eliminate unique variables in arbitrary contexts. It replaces $y \leq u$ by a fresh predicate p_y , and sets the model substitution $y = \text{if } p_y \text{ then } u \text{ else } u - 1$. Conversely, tactics can strengthen formulas. The tactic `symmetry-reduce` applies symmetry reduction for formulas over uninterpreted functions. For illustrative purposes we use a tactic that applies symmetry reduction for arithmetical variables x, y, z that occur symmetric in the formula. Z3 does not contain symmetry reduction tactics with arithmetic so users that produce formulas with symmetries should apply their own symmetry breaking methods. Over-

constrained tactics cannot be applied incrementally if the strengthening formulas are added to the set of assertions of the goal.

8.4. Pre-processing tactics

Tactics are used internally to provide best-effort pre-processing for a set of known benchmark classes. For example, the `qfufbv` tactic applies a pre-amble comprising of several steps, some generic, others that are specific to bit-vector formulas.

```
(and-then simplify propagate-values bv-bound-checks solve-eqs
  elim-uncnstr bv-size-reduction max-bv-sharing ackermannize_bv)
```

We have to refer to [the online guide](#) for an updated description of these tactics and what they accomplish.

As a rule of thumb there is a set of core pre-processing tactics that apply well across formula types. These are

- `simplify` - applies algebraic simplification
- `propagate-values` - detects equations of the form $t \simeq 5$ and replaces t by the value 5 for other occurrences of t .
- `elim-uncnstr` - removes sub-terms that occur only once and can be solved for.
- `solve-eqs` - eliminates uninterpreted variables by solving equalities.

There are also several general purpose tactics that apply only well in selected settings as they may carry more overhead.

- `ctx-simplify` - eliminates sub-formulas based on a contextual analysis. For example $a \wedge (a \implies b)$ can be reduced to $a \wedge b$ because the second occurrence of a is true in the conjunctive context.
- `demodulator` - extracts rewrite rules from quantified equalities and applies them.

Then there are tactics developed to handle specific kinds of formulas. They allow targeting features of a backend that are better amenable to solving classes of formulas. For example, pure bit-vector formulas can often be solved most efficiently by a dedicated SAT solver. The pre-processing steps for the `QF_UFBV` formulas is designed to convert a goal whenever possible into a format where it does not contain uninterpreted functions and where it uses bit-vectors of smallest length.

The main categories of specialized pre-processing tactics are:

- Bit-vectors - compile formulas into the bit-vector fragment.
- Arithmetic - eliminate non-linear operators and transcendental functions. Compile arithmetic formulas into bit-vectors or cardinality constraints.
 - `fm`
 - `factor`
 - `purify-arith`
- Quantifiers detect macros, eliminate quantifiers.
 - `qe-light`
 - `qe`
- Arrays - eliminate or compile occurrences of arrays into uninterpreted functions
 - `bvarray2uf`

8.5. Proof and unsatisfiable core converters

Some tactics also support proofs and extraction of unsatisfiable cores. To support proof generation, a tactic has to map a proof of the subgoal(s) into a proof of the original goal.

A goal assertion can be tracked for the purpose of extracting unsatisfiable cores. Tactics that support unsatisfiable core extraction have to track dependencies of the original goal. Thus, if two assertions φ_1 and φ_2 are used to infer an assertion ψ , then the dependencies for φ_1 and φ_2 are combined into the dependencies for ψ .

8.6. Incremental tactics - a sneak peek

The classification of model converters allow us to understand when tactics can be applied incrementally. We will consider tactics that just produce a single subgoal. Tactics that are equivalence preserving are directly incremental. Tactics that are rigid constrained can be applied incrementally by substituting in the effect of the model converter on new formulas. Tactics that are under-constrained can be applied incrementally by re-adding formulas that were modified, and retracting the effect of the model converter.

A common user issue is around inconsistent behavior of the solver whether it is used in incremental or non-incremental mode. Under the hood, the difference in behaviors is caused by non-incremental mode using a combination of default tactics to pre-process the (only) formula being solved. Once the solver is used in incremental mode (after a context push or after the first check-sat) Z3 switches to a solver mode that does not use the pre-processing tactics. Specifically, the `solve-eqs` and `elim-uncnstr` tactics can have dramatic effect on solvability over many user scenarios and they are not incremental.



As a sneak peek into functionality in the works, configuring z3 with `sat.smt=true` from the command-line switches to a new CDCL(T) core where a set of pre-set tactics, including `solve-eqs` and `elim-uncnstr` are applied incrementally.

9. Interfacing with Z3

As we outlined in the introduction, users of Z3 are likely less interested in the finer details of internal representations than how to make Z3, whatever is under the hood, work better for their applications. The impatient reader might have already given up going over the internals, or just skipped to this section to learn about secret recipes for best using Z3.

In the previous sections, we have included material in an effort to address questions such as:

- how do solvers interact with the core?
- can I control models (to be diverse, random)?
- how can I determine which strategies apply to my formulas?

But we have not given focused answers to questions that cut to the chase of using Z3 to make informed decisions on solving formulas. The purpose of this section is to touch on some broad themes around using Z3 effectively with an eye on how the internals affect using Z3.

9.1. Feeding formulas into Z3

The first layer of interaction with SMT solvers is invariably about how to formulate problems in a way that they match better tuned back-ends. Formula encoding is arguably a source for where the most speedup can be gained. This experience is shared with constraint programming (CP) and mixed-integer programming (MIP) solvers. The SMT format offers some flexibility in encoding formats (you can write disjunctions without introducing big M variables on your own), but using all features is not necessarily an advantage.

We here mention a few anecdotal examples of encoding scenarios.

Using string constants to represent cases of a finite domain is not great.

The string and sequence solver in Z3 is tailored for solving string equations. It has no special handling when it only processes equations of the form $x \simeq "abc"$ that comprise a variable x and a string constant. Applications should not use strings in this case. They can use a finite enumeration sort or bit-vectors.

Solving Sudoku problems with integers is far worse than with bit-vectors.

Sudoku problems have natural formalization using integers. They really only require the distinct constraint over a finite domain $1..9$. The SAT solver is much more effective at solving such constraints (the all-difference constraint is not built into Z3, so has to be programmed [externally if it is useful](#)).

Stick with integers or bit-vectors. Mixed formulas are not solved efficiently.

There are functions for converting integers to bit-vectors (`int2bv`) and from bit-vectors to integers (`bv2nat`). The trigger creating axioms that capture the meaning of these functions. The axioms incur overhead both for bit-vector reasoning and integer reasoning. The solvers decoupled and don't integrate strong propagation for mixed integer/bit-vector reasoning.

Adding constraints that break symmetries tends to have a dramatic effect on problems with symmetries.

Symmetries are a major obstacle to CDCL and essentially all T solvers. There is limited built-in support for breaking symmetries of problems over EUF, but otherwise applications should carefully encode problems in a way that resolve symmetries.

9.2. Controlling search and output behavior

9.2.1. Limitations of CDCL

SAT is a solved problem, right?

Z3 integrates a reasonably efficient SAT solver. There are more efficient modern SAT solvers, specifically Kissat is available for one-shot problems and CaDiCaL supports incrementality. While there can be a significant gap between problems solved by Kissat and Z3's SAT engine on SAT problems, they share some underlying principles that have known limitations.

- CDCL is horrible at xor reasoning. By nature, CDCL SAT searches for regular resolution proofs. Applications that produce parity constraints tend to not have any short regular resolution proofs. A domain solver, based on CryptoMiniSAT, for xor constraints is currently being integrated as a theory solver. Yet, even with native xor reasoning, the art of synthesizing strong lemmas during xor search remains an open problem.



- Z3 is more stable for unsatisfiable problems than satisfiable problems. CDCL solvers are particularly good at finding small resolution proofs. Favoring phase caching (e.g., `smt.phase_caching_on=80000`) is observed to help on satisfiable problems. Tuning of phase caching is among one of the recent advances in Kissat and Z3 borrows some of the ideas.

Parallel Z3 doesn't offer any speedups

- For problems expected with a satisfiable outcome, you can often with some advantage activate parallel SAT solving within the SAT core by setting `sat.threads=N`. It spawns N threads that work on the same problem. Running separate instances with different random seeds may also reduce overall latency in getting answers.
- There is an option to run solvers in parallel by *cube and conquer* decomposition. The idea of cube and conquer is to partition a problem based on one or more case splits (cubes). The cubes are found using a lookahead solver that measures and estimates the benefits of case splitting on Boolean variables. Cube and conquer solving has been used successfully for solving *small* but hard combinatorial problems. A sweet spot for cube and conquer is to decompose problems into more constrained partitions that each have small resolution proofs. Z3's lookahead solver is exposed over the API. It can be used to produce cubes and integrated in a specialized solver. It has shown to be useful also on decomposing larger combinatorial problems solved in a distributed cloud environment. Note that Z3 does not offer any automatic guidance for tuning the ratio and frequency of cubing to conquering and the default settings are unlikely to produce speedups, and much more likely to slow down overall solving.

9.2.2. Ackermann reductions

Shifting reasoning about uninterpreted functions to the CDCL SAT core can produce significant speedups

Z3 contains both pre-processing and in-processing techniques for Ackermann reduction. It replaces uninterpreted functions by propositional logic. Ackermann reduction incurs significant asymptotic overhead, quadratic in the number of a function occurrence, so it is only useful when the number of function occurrences is small (in the tens). The EUF decision procedure is much more scalable in general.

Shifting reasoning about uninterpreted functions to the CDCL SAT core is not practical for large formulas

9.2.3. Converting arithmetic to propositional reasoning

Shifting work to the CDCL SAT core can produce significant speedups

There are several tactics that support transformations from integer arithmetic to propositional reasoning.

- `lia2pb` from bounded arithmetic to Pseudo-Boolean formulas
- `card2bv` from Pseudo-Boolean formulas to bit-vector formulas or other circuit encodings
- `dt2bv` from finite enumeration sorts to bit-vectors
- `bit-blast` from bit-vectors to propositional formulas

Bit-vectors are a convenient intermediary representation that retains equality reasoning or can be fully converted to SAT. The CDCL(T) cores also reason natively with cardinality and Pseudo-Boolean formulas. Native reasoning is an advantage when the clausal forms incur a significant overhead, otherwise, compiling to clausal form very often offers superior performance.

Compiling to SAT can produce significant slowdowns

Conversion into Pseudo-Boolean constraints or bit-vectors is a disadvantage when constraints are mostly convex and can be solved using dual Simplex. The native solver for Pseudo-Booleans in Z3 requires compiling bounded variables into a set of 0-1 variables, and it does not integrate dual Simplex reasoning (yet).

9.2.4. Arithmetic

Z3 subsumes LP and MIP, I should be able to use it for solving LP and MIP problems

Z3's solvers for arithmetic are arguably trained on the instances submitted by users and available in the SMTLIB2 benchmark set. It uses a dual Simplex tableau where constants are represented using infinite precision arithmetic. There are no specialized representations using floating points for fixed-point arithmetic that can be significantly faster for important applications. Reasonable linear programming solvers uses floating point representations. Infinite precision incurs an impractical overhead on standard linear programming applications. On the other hand, there is very little overhead incurred of infinite precision arithmetic for a significant set of applications where Z3 is trained. It *can be significant* however, on important application scenarios such as Solidity smart contracts that involve very large constants.

There are more reals than integers, so isn't search more constrained over integers?

Problems don't get easier because you formulate them as integer problems instead of reals. Quantified formulas over reals polynomials are decided using the `nlsat` tactic. On the other hand, the quantifier free theory of integer polynomials is undecidable by Matiyasevich's theorem.

I want to solve formulas using trigonometric functions and exponents. Z3 allows such formulas so why doesn't it work?

There is no reasoning support for trigonometric functions and exponentiation. Z3's API supports calculation with extension fields over algebraic numbers, a feature that is disjoint from any of the solver capabilities.

9.2.5. Programming Quantifiers

Encoding problems with quantified formulas is extensively studied by users in the programming verification space. A thorough overview of trouble-shooting Z3 behavior for formulas using quantifiers is provided in the context of F^* .

Z3 creates a lot of instantiations. How can I diagnose when there are too many quantifier instantiations?

Z3 fails to instantiate a quantifier I thought would be useful. How do I diagnose a failed quantifier instantiation?

How does Z3 prioritize quantifier instantiation?

9.3. Extracting information from Z3

The primary means for retrieving information from z3 is by extracting *models*, unsatisfiable *cores*, *proof* terms, or clause traces. Clause traces can be stored to a file or monitored directly during search. When run from the command-line you can specify to create a trace log. We summarize the methods below.

Interface	Purpose
model	Retrieve an interpretation of a satisfiable formula Set <code>smt.candidate_models=true</code> to enable model production even if search is incomplete
core	Retrieve an unsatisfiable subset for the <i>assumption</i> literals passed to the solver The cores can optionally be minimized
proof	Retrieve a proof term that tracks main inferences Proof terms are supported for a limited set of pre-processing rules
clause trace	Extract a DRUP(T) style clause trail from search It is relatively new functionality and subject to extensions and revisions
trace	Log a low level trace that can be processed using the <i>AxiomProfiler</i> from ETHZ
API log	Instrument the solver to log API calls into a sequence of instructions that can be replayed
SMT log	Instrument a solver exposed over the API to log assertions, push, pop as SMTLIB2 commands
Lemma log	Instrument a solver to append to standard output a set of learned lemmas in SMTLIB2 format

For development and debugging purposes, you can configure Z3 in *verbose* mode and instrument Z3's code with low level tracing.

The guide on programming quantifiers 9.2.5 points to detailed information of using feedback from Z3 to understand search behavior as a user.

10. Summary

This document developed a bird's eye view of Z3 internals. It offered some ideas of the data-structures and algorithms used at a level to capture the gist in the format of an extended tool description. It also provided background on how users can interact with Z3 to develop encodings and operate the solver. A vast amount of details is omitted from this document. Some are described in papers that originally introduced the features, other details have yet to be distilled from implementations into a literate format.

References

- Mislav Balunovic, Pavol Bielik, and Martin T. Vechev. Learning to solve SMT formulas. In *NeurIPS 2018*, 2018. URL <https://proceedings.neurips.cc/paper/2018/hash/68331ff0427b551b68e911eebe35233b-Abstract.html>. 
- Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The Satisfiability Modulo Theories Library (SMT-LIB), 2016. URL <http://smtlib.cs.uiowa.edu>. 
- Nikolaj Bjørner. Linear quantifier-elimination as an abstract decision procedure. In *IJCAR*, 2010. doi:[10.1007/978-3-642-14203-1_27](https://doi.org/10.1007/978-3-642-14203-1_27). 
- Nikolaj Bjørner and Mikolás Janota. Playing with alternating quantifier satisfaction. In *LPAR Short presentation papers*, 2015. doi:[10.29007/vv21](https://doi.org/10.29007/vv21). 

- Nikolaj Bjørner and Lev Nachmanson. Theorem recycling for theorem proving. In *Vampire*, 2017. doi:[10.29007/r58f](https://doi.org/10.29007/r58f). 
- Nikolaj Bjørner, Kenneth L. McMillan, and Andrey Rybalchenko. On solving universally quantified horn clauses. In *SAS*, 2013. doi:[10.1007/978-3-642-38856-9_8](https://doi.org/10.1007/978-3-642-38856-9_8). 
- Nikolaj Bjørner, Arie Gurfinkel, Kenneth L. McMillan, and Andrey Rybalchenko. Horn clause solvers for program verification. In *Fields of Logic and Computation II - Essays Dedicated to Yuri Gurevich on the Occasion of His 75th Birthday*, 2015. doi:[10.1007/978-3-319-23534-9_2](https://doi.org/10.1007/978-3-319-23534-9_2). 
- Nikolaj Bjørner, Leonardo de Moura, Lev Nachmanson, and Christoph M. Wintersteiger. Programming Z3. In *SETSS*, 2018. doi:[10.1007/978-3-030-17601-3_4](https://doi.org/10.1007/978-3-030-17601-3_4). 
- Nikolaj S. Bjørner and Lev Nachmanson. Navigating the universe of Z3 theory solvers. In *SBMF*, 2020. doi:[10.1007/978-3-030-63882-5_2](https://doi.org/10.1007/978-3-030-63882-5_2). 
- Maria Paola Bonacina, Christopher Lynch, and Leonardo Mendonça de Moura. On deciding satisfiability by theorem proving with speculative inferences. *J. Autom. Reasoning*, 47 (2): 161–189, 2011. doi:[10.1007/s10817-010-9213-y](https://doi.org/10.1007/s10817-010-9213-y). 
- Aaron R. Bradley, Zohar Manna, and Henny B. Sipma. What's decidable about arrays? In *VMCAI*, 2006. doi:[10.1007/11609773_28](https://doi.org/10.1007/11609773_28). 
- Martin Bromberger and Christoph Weidenbach. Fast cube tests for LIA constraint solving. In *IJCAR*, 2016. doi:[10.1007/978-3-319-40229-1_9](https://doi.org/10.1007/978-3-319-40229-1_9). 
- Martin Bromberger and Christoph Weidenbach. New techniques for linear arithmetic: cubes and equalities. *Formal Methods in System Design*, 51 (3): 433–461, 2017. doi:[10.1007/s10703-017-0278-7](https://doi.org/10.1007/s10703-017-0278-7). 
- Jürgen Christ and Jochen Hoenicke. Cutting the mix. In *CAV*, 2015. doi:[10.1007/978-3-319-21668-3_3](https://doi.org/10.1007/978-3-319-21668-3_3). 
- Alessandro Cimatti, Alberto Griggio, Ahmed Irfan, Marco Roveri, and Roberto Sebastiani. Experimenting on solving nonlinear integer arithmetic with incremental linearization. In *SAT*, 2018. doi:[10.1007/978-3-319-94144-8_23](https://doi.org/10.1007/978-3-319-94144-8_23). 
- George E. Collins. Quantifier elimination for real closed fields by cylindrical algebraic decomposition—preliminary report. *SIGSAM Bull.*, 8 (3): 80–90, 1974. doi:[10.1145/1086837.1086852](https://doi.org/10.1145/1086837.1086852). 
- Leonardo Mendonça de Moura and Nikolaj Bjørner. Generalized, efficient array decision procedures. In *FMCAD*, 2009. doi:[10.1109/FMCAD.2009.5351142](https://doi.org/10.1109/FMCAD.2009.5351142). 
- Leonardo Mendonça de Moura and Grant Olney Passmore. The strategy challenge in SMT solving. In *Automated Reasoning and Mathematics - Essays in Memory of William W. McCune*, pages 15–44, 2013. doi:[10.1007/978-3-642-36675-8_2](https://doi.org/10.1007/978-3-642-36675-8_2). 
- Leonardo Mendonça de Moura and Nikolaj Bjørner. Efficient e-matching for SMT solvers. In *CADE*, 2007. doi:[10.1007/978-3-540-73595-3_13](https://doi.org/10.1007/978-3-540-73595-3_13). 
- Leonardo Mendonça de Moura and Nikolaj Bjørner. Bugs, moles and skeletons: Symbolic reasoning for software development. In *IJCAR*, 2010. doi:[10.1007/978-3-642-14203-1_34](https://doi.org/10.1007/978-3-642-14203-1_34). 
- David Detlefs, Greg Nelson, and James B. Saxe. Simplify: a theorem prover for program checking. *J. ACM*, 52 (3): 365–473, 2005. doi:[10.1145/1066100.1066102](https://doi.org/10.1145/1066100.1066102). 
- Isil Dillig, Thomas Dillig, and Alex Aiken. Cuts from proofs: A complete and practical technique for solving linear inequalities over integers. In *CAV*, 2009. doi:[10.1007/978-3-642-02658-4_20](https://doi.org/10.1007/978-3-642-02658-4_20). 
- Peter J. Downey, Ravi Sethi, and Robert Endre Tarjan. Variations on the common subexpression problem. *J. ACM*, 27 (4): 758–771, 1980. doi:[10.1145/322217.322228](https://doi.org/10.1145/322217.322228). 
- B. Dutertre and L. de Moura. A Fast Linear-Arithmetic Solver for DPLL(T). In *CAV*, 2006. doi:[10.1007/11817963_11](https://doi.org/10.1007/11817963_11). 
- Oliver Flatt, Samuel Coward, Max Willsey, Zachary Tatlock, and Pavel Panchekha. Small Proofs from Congruence Closure. In *FMCAD*, 2022. 
- Yeting Ge and Leonardo Mendonça de Moura. Complete instantiation for quantified formulas in satisfiability modulo theories. In *CAV*, 2009. doi:[10.1007/978-3-642-02658-4_25](https://doi.org/10.1007/978-3-642-02658-4_25). 
- Sergey Grebenshchikov, Nuno P. Lopes, Corneliu Popeea, and Andrey Rybalchenko. Synthesizing software verifiers from proof rules. In *PLDI*, 2012. doi:[10.1145/2254064.2254112](https://doi.org/10.1145/2254064.2254112). 
- Arie Gurfinkel. Program verification with constrained horn clauses (invited paper). In *CAV*, 2022. doi:[10.1007/978-3-031-13185-1_2](https://doi.org/10.1007/978-3-031-13185-1_2). 
- Krystof Hoder and Nikolaj Bjørner. Generalized property directed reachability. In *SAT*, 2012. doi:[10.1007/978-3-642-31612-8_13](https://doi.org/10.1007/978-3-642-31612-8_13). 
- Bart Jacobs. *Categorical Logic and Type Theory*, volume 141 of *Studies in Logic and the Foundations of Mathematics*. North Holland, Elsevier, 1999. doi:[10.2307/421214](https://doi.org/10.2307/421214). 
- Ajith K. John and Supratik Chakraborty. A quantifier elimination algorithm for linear modular equations and disequations. In *CAV*, 2011. doi:[10.1007/978-3-642-22110-1_39](https://doi.org/10.1007/978-3-642-22110-1_39). 
- Ajith K. John and Supratik Chakraborty. Extending quantifier elimination to linear inequalities on bit-vectors. In *TACAS*, 2013. doi:[10.1007/978-3-642-36742-7_6](https://doi.org/10.1007/978-3-642-36742-7_6). 
- Ajith K. John and Supratik Chakraborty. A layered algorithm for quantifier elimination from linear modular constraints. *Formal Methods in System Design*, 49 (3): 272–323, 2016. doi:[10.1007/s10703-016-0260-9](https://doi.org/10.1007/s10703-016-0260-9). 
- Dejan Jovanovic and Leonardo Mendonça de Moura. Solving non-linear arithmetic. In *IJCAR*, 2012. doi:[10.1007/978-3-642-31365-3_27](https://doi.org/10.1007/978-3-642-31365-3_27). 
- Deepak Kapur and Calogero Zarba. A reduction approach to decision procedures. Technical report, University of New Mexico, 2006. URL <https://www.cs.unm.edu/~kapur/mypapers/reduction.pdf>. 
- Nuno P. Lopes, Junyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *PLDI*, 2021. doi:[10.1145/3453483.3454030](https://doi.org/10.1145/3453483.3454030). 
- Shin-ichi Minato. Zero-suppressed bdds for set manipulation in combinatorial problems. In Alfred E. Dunlop, editor, *DAC*, 1993. doi:[10.1145/157485.164890](https://doi.org/10.1145/157485.164890). 
- Robert Nieuwenhuis and Albert Oliveras. Fast congruence closure and extensions. *Inf. Comput.*, 205 (4): 557–580, 2007. doi:[10.1016/j.ic.2006.08.009](https://doi.org/10.1016/j.ic.2006.08.009). 
- Masaaki Nishino, Norihito Yasuda, Shin-ichi Minato, and Masaaki Nagata. Zero-suppressed sentential decision diagrams. In *AAAI*, 2016. URL <http://www.aaai.org/ocs/index.php/AAAI/AAAI16/paper/view/12434>. 
- Grant O. Passmore, Simon Cruanes, Denis Ignatovich, Dave Aitken, Matt Bray, Elijah Kagan, Kostya Kanishev, Ewen Maclean, and Nicola Mometto. The imandra automated reasoning system (system

description). In *IJCAR*, 2020. doi:[10.1007/978-3-030-51054-1_30](https://doi.org/10.1007/978-3-030-51054-1_30). 

Anh-Dung Phan, Nikolaj Bjørner, and David Monniaux. Anatomy of alternating quantifier satisfiability (work in progress). In *SMT*, 2012. doi:[10.29007/8cf7](https://doi.org/10.29007/8cf7). 

Martina Seidl, Florian Lonsing, and Armin Biere. qbf2epr: A tool for generating EPR formulas from QBF. In *PAAR*, 2012. URL <http://www.easychair.org/publications/paper/145184>. 

Caleb Stanford, Margus Veanes, and Nikolaj Bjørner. Symbolic boolean derivatives for efficiently solving extended regular expression constraints. Technical Report MSR-TR-2020-25, Microsoft, August 2020. URL <https://www.microsoft.com/en-us/research/publication/symbolic-boolean-derivatives-for-efficiently-solving-extended-regular-expression-constraints/>. 

Alfred Tarski. A decision method for elementary algebra and geometry. 1948. doi:[10.1007/978-3-7091-9459-1_3](https://doi.org/10.1007/978-3-7091-9459-1_3). 

Christoph M. Wintersteiger, Youssef Hamadi, and Leonardo Mendonça de Moura. Efficiently solving quantified bit-vector formulas. *Formal Methods in System Design*, 42 (1): 3–23, 2013. doi:[10.1007/s10703-012-0156-2](https://doi.org/10.1007/s10703-012-0156-2). 

Created with [Madoko.net](https://madoko.net/).

-
1. To keep the formula short, we have applied a shortcut and removed the literal $\neg(x + y < 0)$ from the conflict clause. 
 2. thanks to Jasmin Blanchette for drawing attention to this distinction. 
 3. Thanks to Simon Cuares for initiating the implementation for recursive functions. 

